Fachhochschule Nordwestschweiz
Hochschule für Technik

Software Architecture

Architecture Means – Architecture Patterns

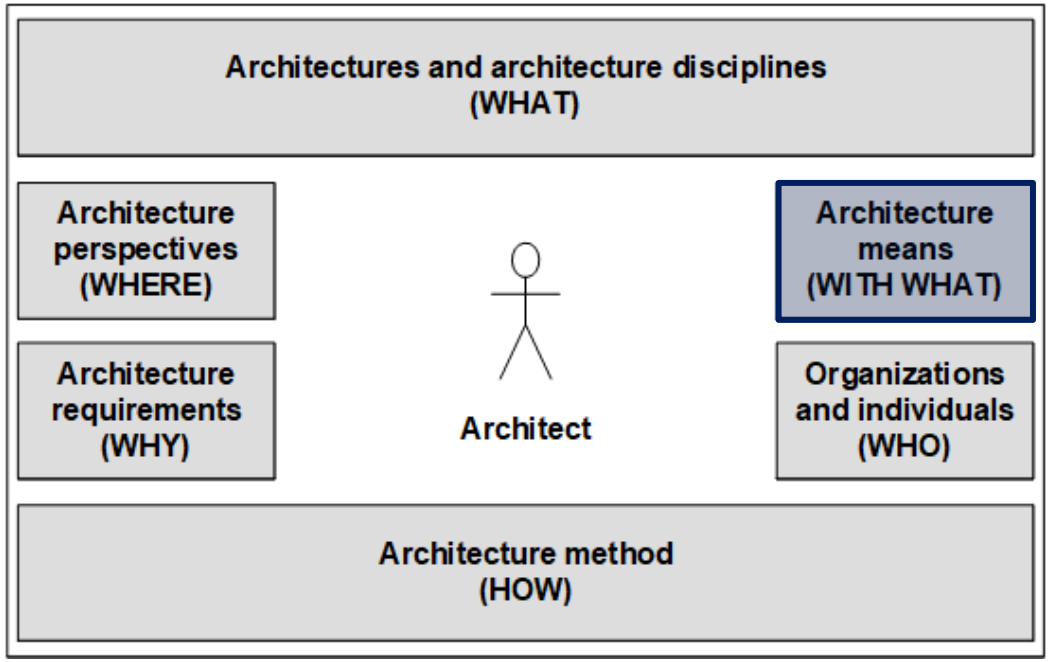
BSc

Ingo Arnold

Copyright Notice

Lecture Opening
Architecture Orientation Framework

Architecture **WITH WHAT discusses the broad range of architectural means architects use to plan, develop, and operate architectures**. Thus what we consider here is the architect's toolbox, if you wish. [Vogel, Arnold et al 2011].



Lecture Opening
Motivation

Diagrams become confusing when the number of represented components (classes, modules, etc.) increases.

Humans can store 7 plus/minus 2 elements in short-term memory. If the amount of information units exceeds this, we reach our overview limit.

Therefore e.g. **class diagrams (even for smaller systems) become overwhelming quite quickly**.

Components and modularization are an option to abstract from single classes and to summarize them in coarser units. These coarsened structures are again easier to understand.

At the same time the coarser abstraction reduces the possibilities for relations (in the development view) between classes, since two classes from different components can be dependent on each other only if a relation exists between these components.

Lecture Opening
Motivation

Architecture patterns are another option for introducing structure and order such that oversight of a system is maintained.

Lecture Opening
Learning Objectives

You …

- understand the **nature and purpose of patterns and pattern languages**.
- understand **similarities as well as differences between architecture patterns and software design patterns** and can explain them.
- can **recognize, name and describe the most important architecture patterns**.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

Pattern
Nature and Purpose

Pattern is a **pivotal concept for architecture**. They are all around us, from the people we meet to the repetitive patterns we find in nature and daily routine.

A pattern **abstracts and generalizes a concrete phenomenon** so that it represents a whole class of concrete phenomena. To recognize a pattern is to recognize commonalities among concrete phenomena.

As a means of abstraction, a pattern is an **excellent vehicle for documenting and exchanging know-how and expertise**.

Patterns provide a **common vocabulary, language, and protocol** that resolves or mitigates ambiguities in exchanges between communicating parties.

Patterns also **provide us with guidance and inform us of the potential consequences** of their use.

Pattern
Nature and Purpose

Patterns **help us efficiently learn and memorize new phenomena** we encounter. They give us the confidence that a situation we have mastered in the past can be mastered again by bringing a proven pattern to bear.

However, while patterns are concrete means for orientation in the world and a means for adaptation of solution approaches to recurring problems, **patterns remain a heuristic**.

From a problem-solving perspective, each **pattern is a recipe that describes what one must do to create the solution that the pattern prescribes**.

Patterns are powerful **means for efficiently finding, discussing, and deciding effective solutions**, while at the same time no pattern ever is the solution.

Problems recur, and so do their corresponding responses. **By generalizing the context, problem, and associated solution, we obtain a reusable solution pattern** – that is, a highly efficient and effective solution blueprint.

Pattern
Nature and Purpose

The **stairs pattern is an example to illustrate the basic pattern idea** and to visualize the ways in which patterns can be used and referenced in real life.

The recurring need (**problem**) that the stairs pattern addresses is that of a person who wants to negotiate the height between floors while walking upright.

example

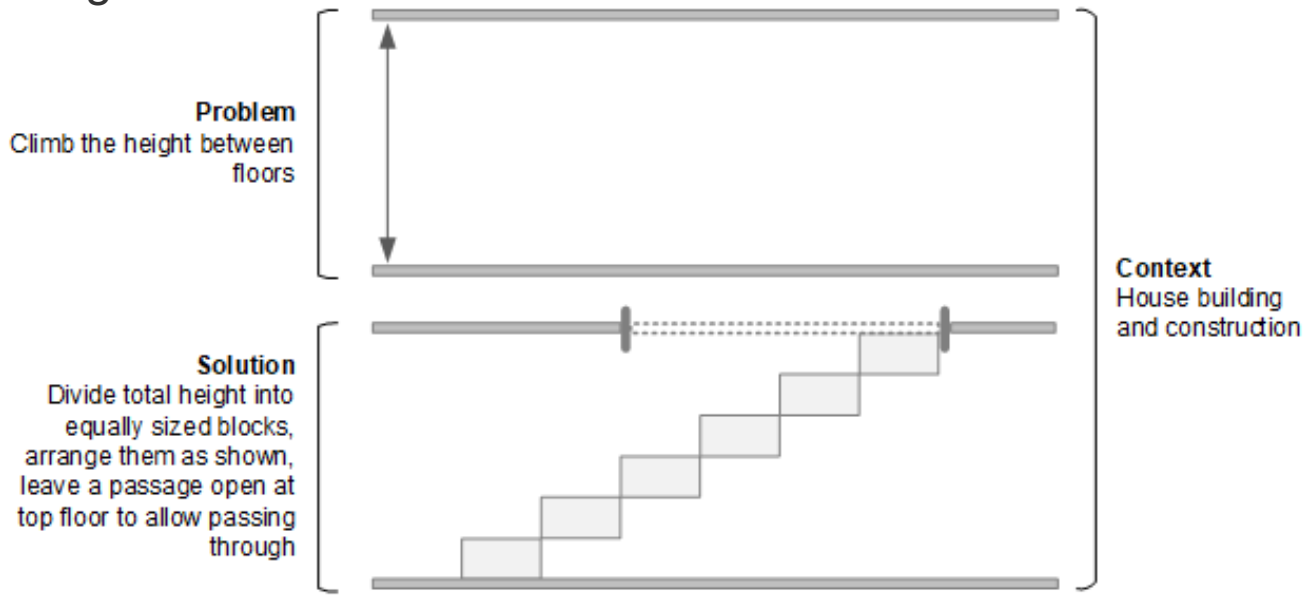
Pattern
Nature and Purpose

What is the stairs pattern’s context and solution approach?

example

Pattern
Nature and Purpose

The **context** in which stairs are planned, discussed, and referenced is house construction. The proven **solution** suggested by the stairs pattern is one in which the total height is divided into blocks of equal size, in which the blocks are arranged as shown, and in which a passageway is left open at the top floor such that a person can pass through

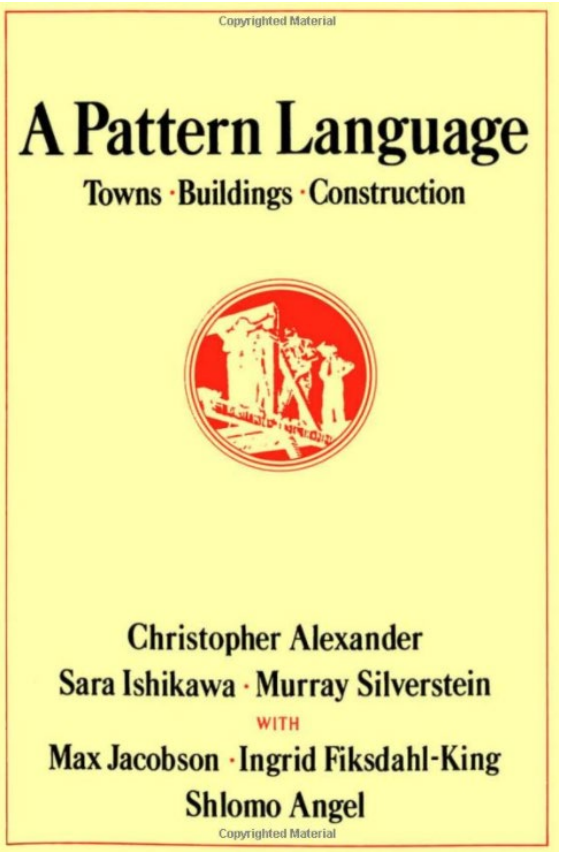


Pattern
History

Patterns have become famous through Christopher Alexander's book *A pattern language* [Alexander 1977].

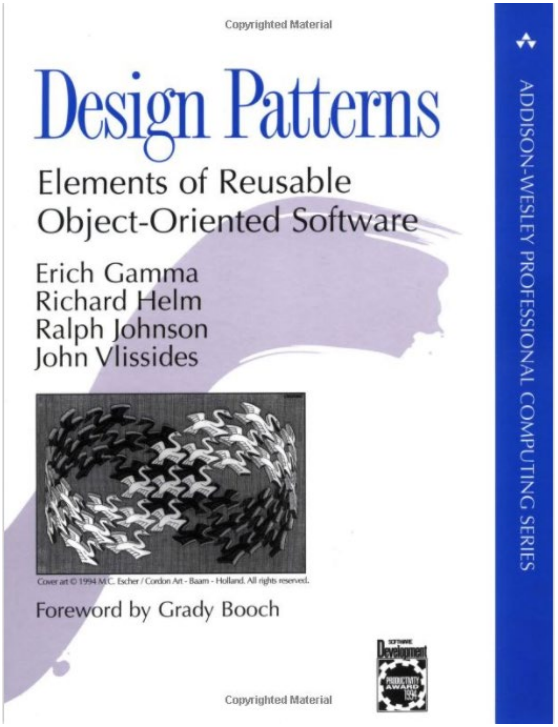
Alexander presented patterns of urban architecture. He defines a pattern as a description of a proven architecture approach associated with a recurring problem in the urban context.

His book is the **first architecture best practice summary in more than 2000 years**, when Marcus Vitruvius Pollio (c. 80-70 BC—after c. 15 BC), commonly known as Vitruvius, published his multi-volume work *De Architectura – Libri Decem* [Vitruvius 2013].



Pattern
History

In the early 1990s, Christopher Alexander's work was **adapted to the domain of software development by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, known as the Gang of Four (GoF)**, in their book *Design Patterns – Elements of Reusable Object-Oriented Software* [Gamma 1994].



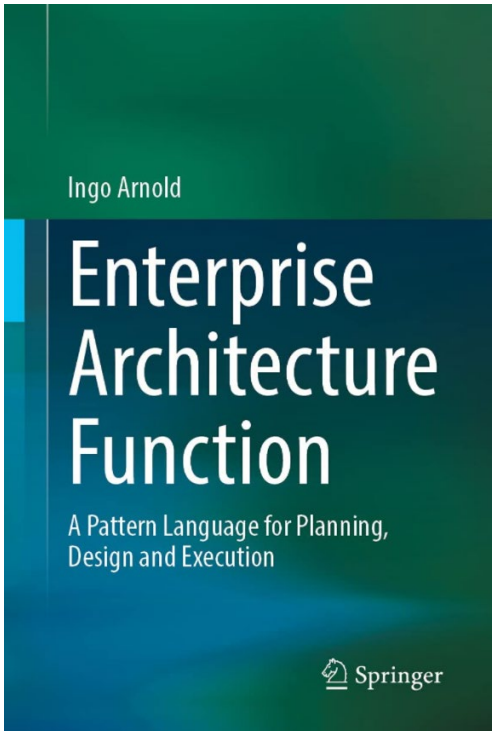
Pattern
History

Since then, many m**ore pattern contributions have emerged that adapt patterns and pattern language as a methodological technique for analyzing and representing classes of solution designs for associated classes of problems**.

For example, **patterns for the analysis of domains** [Fowler 1997], **patterns for domain-driven design** [Evans 2004], or **patterns for enterprise architecture management** [Matthes et al. Pattern Catalogue 2015] ...

Pattern
History

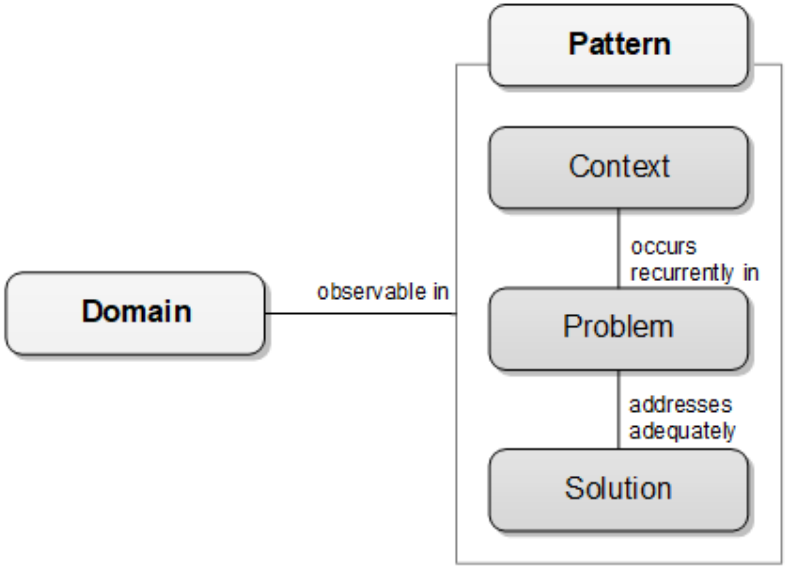
… even patterns for the design of architecture in terms of an organizational capability – thus **patterns for the design of an architecture function** [Arnold 2021].



Pattern
Definition

While there are many differences between the domains for which authors have published pattern books or papers, there is also a fundamental commonality.

As the least common denominator, all pattern publications share the basic pattern triad scheme as introduced by Christopher Alexander.



example

Pattern
Definition

What are the typical attributes along which patterns are described (beyond name, context, problem, and solution)?

example

Pattern
Definition

Detailed pattern descriptions often include the following attributes.

Attribute	Value
Name	The prime name of the pattern
Also known as	Pattern synonyms
Type	Categories like structural, behavioural, creational
Problem	What the pattern sets out to solve - may include a simple concrete example
Context	Context in which the pattern can recurringly be observed
Forces	Forces the pattern needs to balance and respond to
Proposed Solution	Solution approach explained in terms of the concrete example, above
Consequences	What happens when the pattern is applied - good and bad
Related Patterns	References to complementary patterns and a description of how to combine them
Known uses	At least three known uses and applications of the given pattern should exist

Pattern
Definition

Patterns **outline the common characteristics of problem-solving phenomena by generalizing their context, problem, and associated solution**.

In addition, patterns **involve variations in their generalization** of context, problem, and solution.

For example, structural versus behavioural variations, or variations in the degree of abstraction of patterns (e.g., design patterns versus architecture patterns).

This implies that **patterns must be concretized and translated when they are adapted to establish concrete solutions** in response to correspondingly concrete problems.

example

Pattern
Example

Using the **example of two granite houses**, a traditional construction method in the Swiss canton of Ticino, you can nicely see how the previously discussed stairs pattern can be physically realized in very different ways.

While both staircases are equally derived from the stairs pattern, their concrete constructions differ significantly.

Pattern
Application

Beyond prescribing context, problem, and solution state, **inherent in a pattern is the process of its own application** – that is, a pattern is always both a thing and a process.

Patterns **enable the complete reasoning cycle**, from initial orientation (i.e., context), through reflection and understanding of the challenge at hand (i.e., problem), to an adequate approach to resolving the challenging forces (i.e., solution).

Patterns must be made available in such a way that they can be practically found and applied. Therefore, they **must be documented in a structured way, maintained, evolved, promoted, and made publicly available**.

In addition to making patterns available, the **utilization and implementation of patterns must be learned and practiced**.

Finally, **patterns are disseminated and made publicly available through pattern catalogues and pattern languages**.

Pattern
Pattern Language

Alexander also introduced the notion of pattern language, which he likened to ordinary language in terms of its expressive and constructive power:

> *"The people can shape buildings for themselves, and have done it for centuries, by using languages which I call pattern languages. A pattern language gives each person who uses it, the power to create an infinite variety of new and unique buildings, just as his ordinary language gives him the power to create an invite variety of sentences."*

While an ordinary language like English or German allows us to create an infinite variety of word combinations called sentences, **a pattern language is a system that allows its users to create an infinite variety of context-problem-solution triples**.

Pattern
Pattern Language

Just as **there are natural relationships between related problems**, similarly **natural relationships exist between patterns**.

For any reasonably complex real-world problem, **a single pattern is often not enough to suggest a complete solution**.

You need to **find and apply associated patterns** (and associated patterns of associated patterns, and so on) to deal with the many facets of the real-world problem in their entirety.

This is where pattern languages come into play. **While an individual pattern describes a solution to a single problem, a pattern language is a collection of semantically related patterns spanning a larger solution space**.

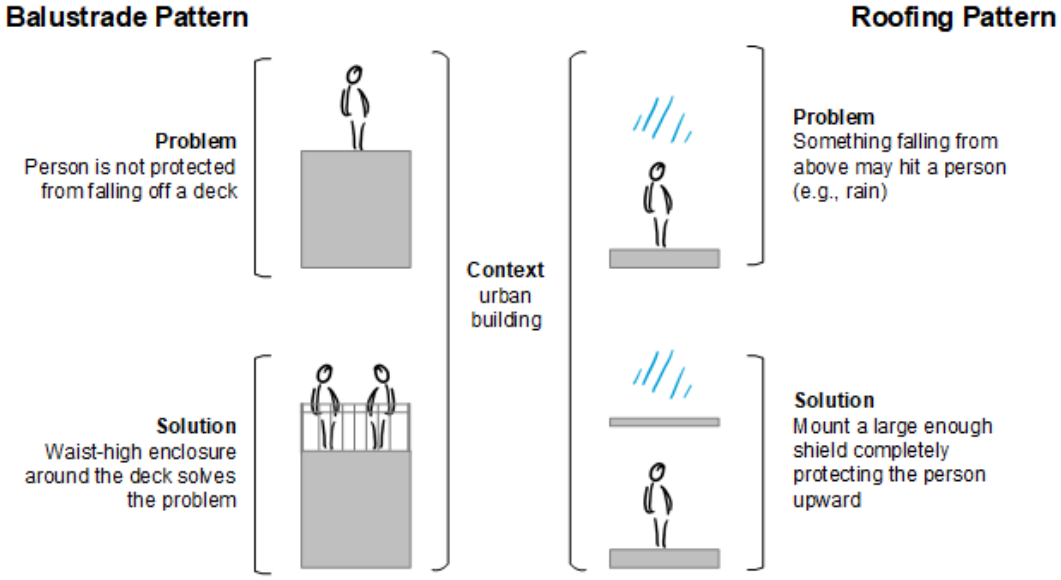
The solution **space addressed by a pattern language is appropriately oriented towards complex and interrelated problems**.

Pattern
Pattern Language Example

Let us look at *balustrade* and *roofing* as complementary patterns to *stairs*.

The *balustrade* problem is one where a person is not protected from falling off a deck, while a waist-high enclosure around the deck adequately solves the problem.

The *roofing* problem is one where something can fall from above and hit a person, while the solution is a large enough shield mounted so that it fully protects the person upward.



example

Pattern

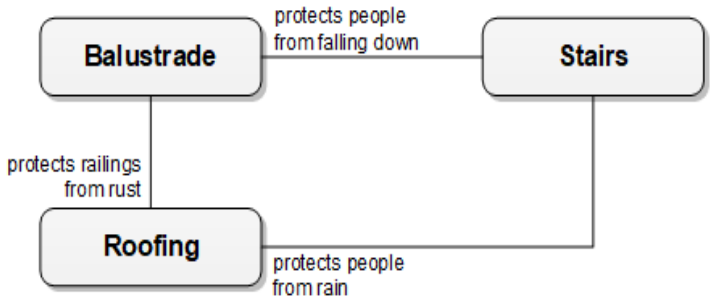
Pattern Language Example

How may a pattern language that relates Balustrade, Stairs, and Roofing look like?

example

Pattern
Pattern Language Example

An **excerpt of the pattern language** may look like this.



When an architect uses the language of *stairs*, *balustrade*, and *roofing* patterns in the context of designing an entire building, an architect may first consider the problem of overcoming the height between floors.

In this way, the architect first finds the *stairs* pattern.

By studying the stairs pattern, the same architect may recognize that stairs above a certain height expose their users to the risk of falling. Fortunately, the pattern language suggests an associated pattern (*balustrade*) to solve the particular problem of falling.

Pattern
Pattern Language Example

The stairs pattern points the architect to another associated pattern (*roofing*).

Since the planning of the new building is still in its infancy, the architect has not yet considered protection from anything that might fall from above.

However, by explicitly referencing the stairs pattern, the architect recognizes that at a later date, a staircase is indeed planned for the exterior of the building.

This will be a moment when *roofing* comes in handy and will address the problem of protecting the stairs users from bad weather conditions, such as rain.

Pattern
Pattern Language

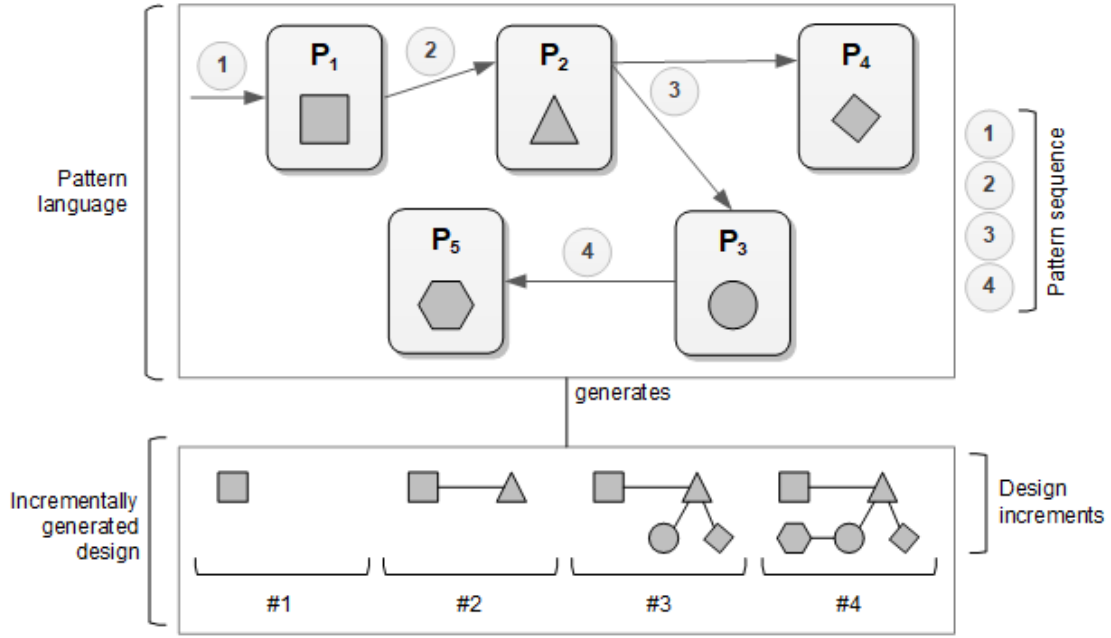
The example of the *stairs*, *balustrade*, and *roofing* demonstrated the value of a pattern language.

A **pattern language provides solutions to problems that none of its individual patterns entirely address**.

Similar to a complex system being broken down into smaller but interrelated parts, **a pattern language, viewed as a system of patterns, is broken down into related patterns**.

Pattern
Pattern Language

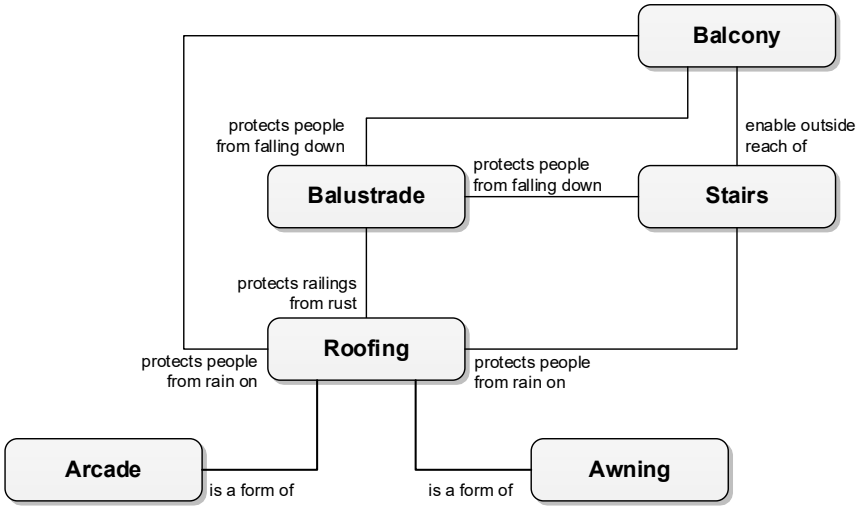
Whether you want to design a building, or a computer system, use an appropriate pattern language as a tool to generate your desired design in an incremental, systematic, and guided manner.

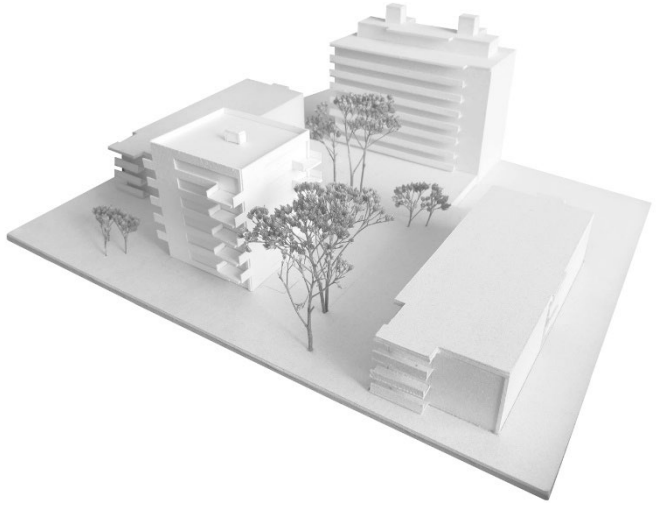


Pattern
Pattern Language

Pattern languages are tools for generating holistic designs in response to complex problems.

Pattern Language for «Building Design»

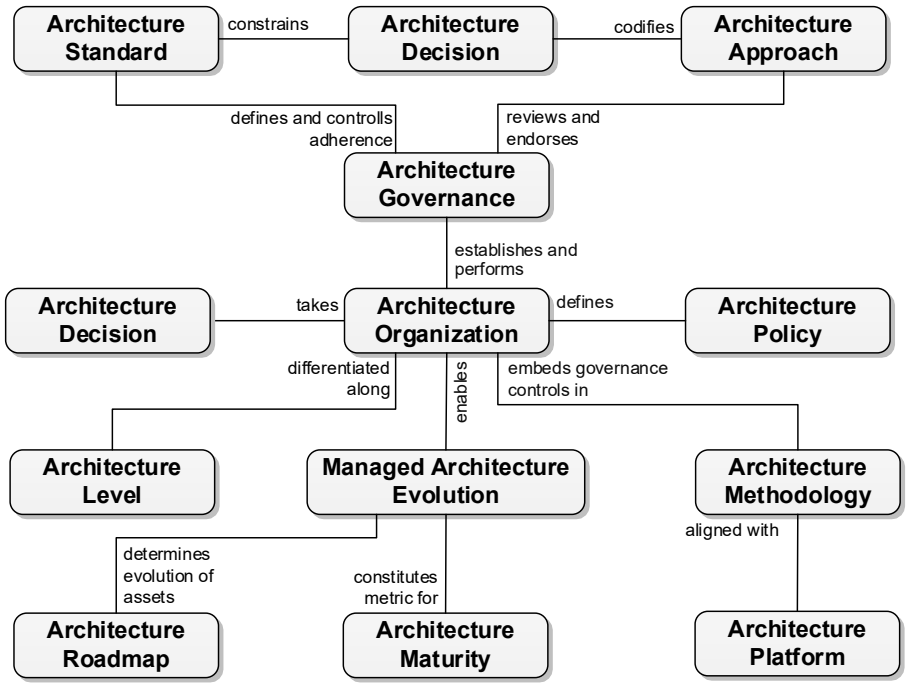


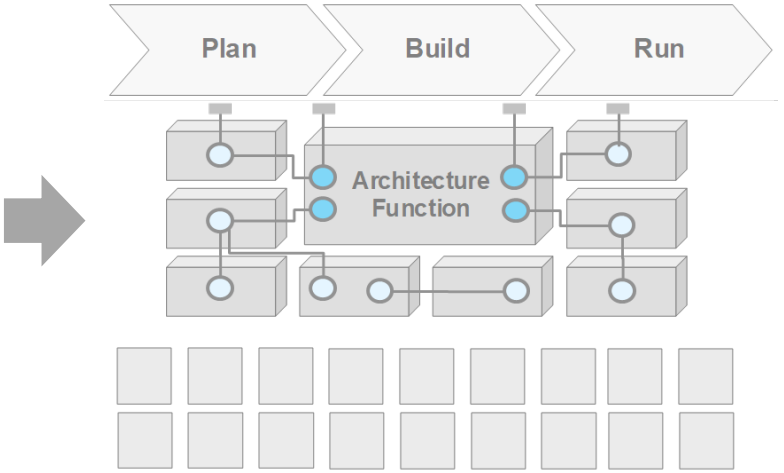


Pattern
Pattern Language

Pattern languages are tools for generating holistic designs in response to complex problems.

Pattern Language for «Enterprise Architecture Function Design»





Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

Architecture Pattern
Overview

As we have already seen, **patterns capture distilled commonalities found in concrete solutions**. A pattern provides a general solution to a class of problems.

A large complex software goes through a series of deconstructions at different levels.

At the coarsest level, **architecture patterns help us make strategic and large-scale design decisions**.

At a finer grained level, **software design patterns are our tools for making rather tactical and smaller-scale design decisions**.

Finally, at the implementation level, **programming paradigms and ultimately programming languages are our tools for making implementation-oriented design decisions**.

Architecture Pattern
Overview

For a very simplified view:

- **Architecture patterns**. Basic structural organization for entire software systems. They are high-level strategies that involve large-scale components, their relationships, global properties and mechanisms of an entire system.
- **Design patterns**. Solve recurring problems in software design and are medium-level tactics that elaborate some of the (internal) structure and behavior of components as well as internal relationships.
- **Programming paradigms and idioms** are language-specific programming techniques that fill in low-level details.

Architecture Pattern
Overview

However, let us remember how Grady Booch [Booch 2009] defined Architecture:

"Architecture is the total of significant design decisions, where significant is measured by cost of change"

This definition suggests that **sometimes a given pattern will qualify as rather *architectural* and sometimes as rather not** (i.e., *as only design*).

This means that **it is not necessarily an intrinsic property of a pattern as to whether it can be considered architectural.** Instead it becomes a property of its application.

Architecture Pattern
Overview

When we talk about **software**, we are referring to **only one aspect** (i.e., a technicality if you wish) **of an overall IT-based solution**.

However, **software architects need to also consider the operational environment** in which their software is executed.

The **(overall) system (i.e., IT system as a whole) and its functional as well as non-functional capabilities are determined by the software that a programmer developed *and* the operating environment into which this software was deployed**.

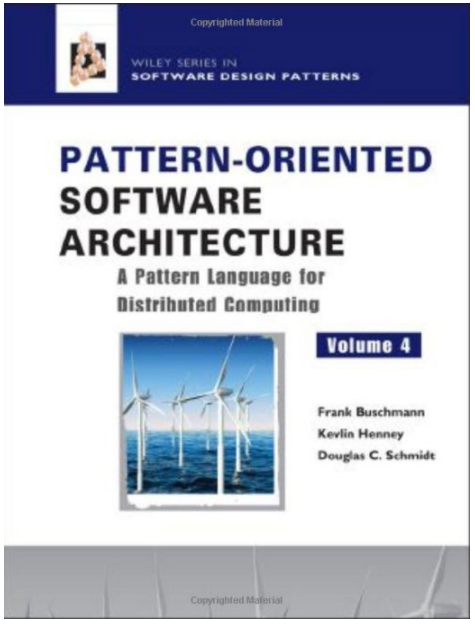
The task of the software architect is to consider and design the IT-based system as a whole.

Architecture patterns address in particular those design issues that concern an IT-based system in its entirety.

Architecture Pattern
Overview

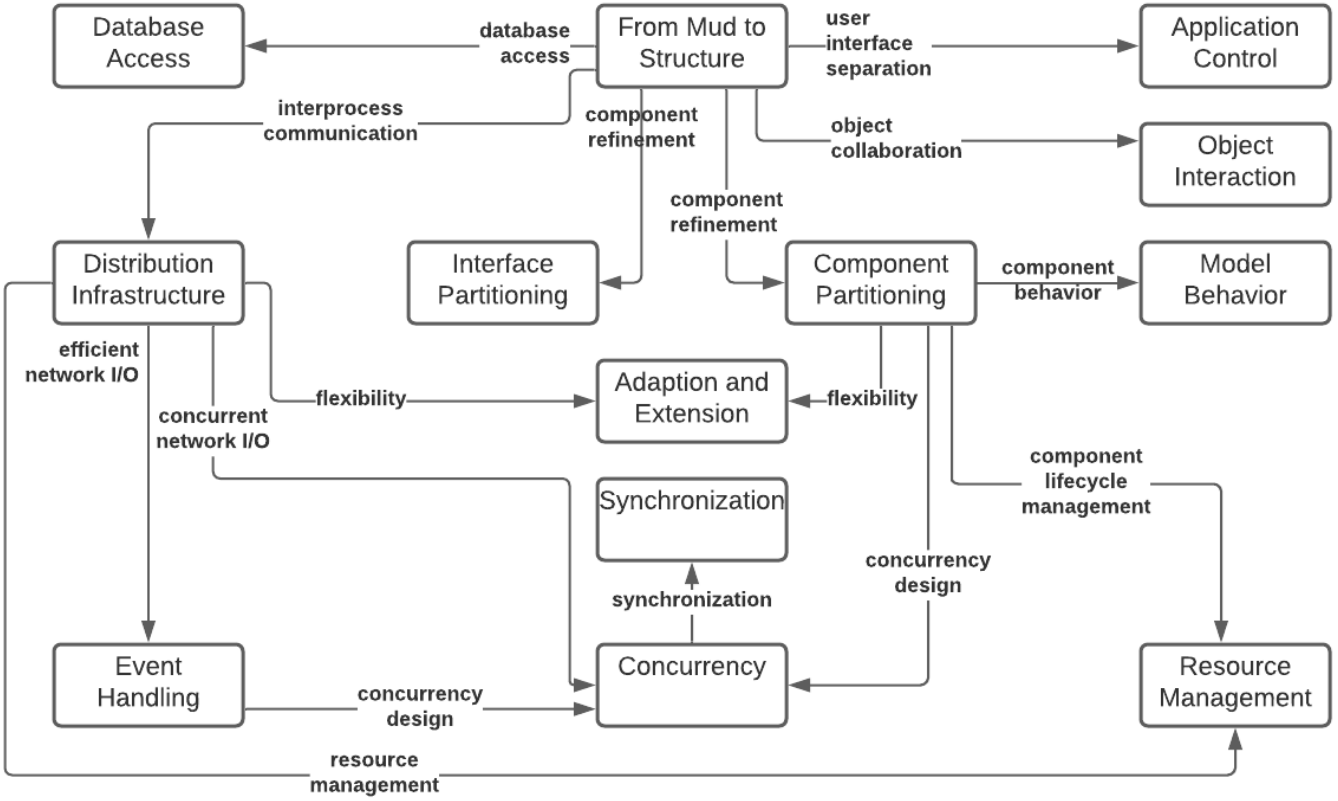
Many architecture patterns (free from a given application context) **have been identified by corresponding authors** and propagated in their publications.

For example, Frank Buschmann [Buschmann et al 2007] et al propose an **Architecture Pattern Language for distributed systems**.



Architecture Pattern
Overview

The top-level **order of the pattern language consists of the important and recurring areas of concern that every distributed system faces**.



Architecture Pattern
Overview

Which of these patterns have we already discussed?

Within each area of concern the authors identify a variety of architecture patterns addressing respective concerns of distributed systems.

Area of Concern	Architecture Patterns
From Mud to Structure	Layers, Model-View-Controller, Microkernel, Reflection, Pipes and Filters, Shared Repository, ...
Distribution Infrastructure	Messaging, Broker, Publish-Subscriber, Client Proxy, Client Request Handler, ...
Interface Partioning	Explicit Interface, Introspective Interface, Facade, Iterator, Batch Method, ...
Application Control	Front Controller, Firewall Proxy, Authorization, ...
Adaption and Extension	Interceptor, Interpreter, Template Method, Strategy, Decorator, ...
Resource Management	Component Configurator, Lookup, Virtual Proxy, Resource Pool, Lazy Acquisition, Automated Garbage Collection, Factory Method, ...

Architecture Pattern
Overview

In this lecture, we will take a closer look at the following **Architecture Patterns**:

- From Mud to Structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection
- Distribution Infrastructure
  - Messaging
  - Publish-Subscribe
  - Broker
  - Container
  - Virtual Proxy
  - Lifecycle Callback
  - Resource Pool

Architecture Pattern
Overview

In this lecture, we will take a closer look at the following **Architecture Patterns**.

- Database Persistence
  - Database Access Layer
  - Data Mapper
- Security
  - Firewall Proxy
  - Authorization
- Other
  - Microservices-based Architecture

Architecture Pattern
Overview

The **patterns that we will discuss are associated with other patterns**.

Associated patterns I will always present at the beginning of a pattern introduction (i.e., in the form of a **pattern language excerpt**).

Some of the associated patterns we have already met as software design patterns.

Other associated patterns are architecture patterns and we discuss them in this deck.

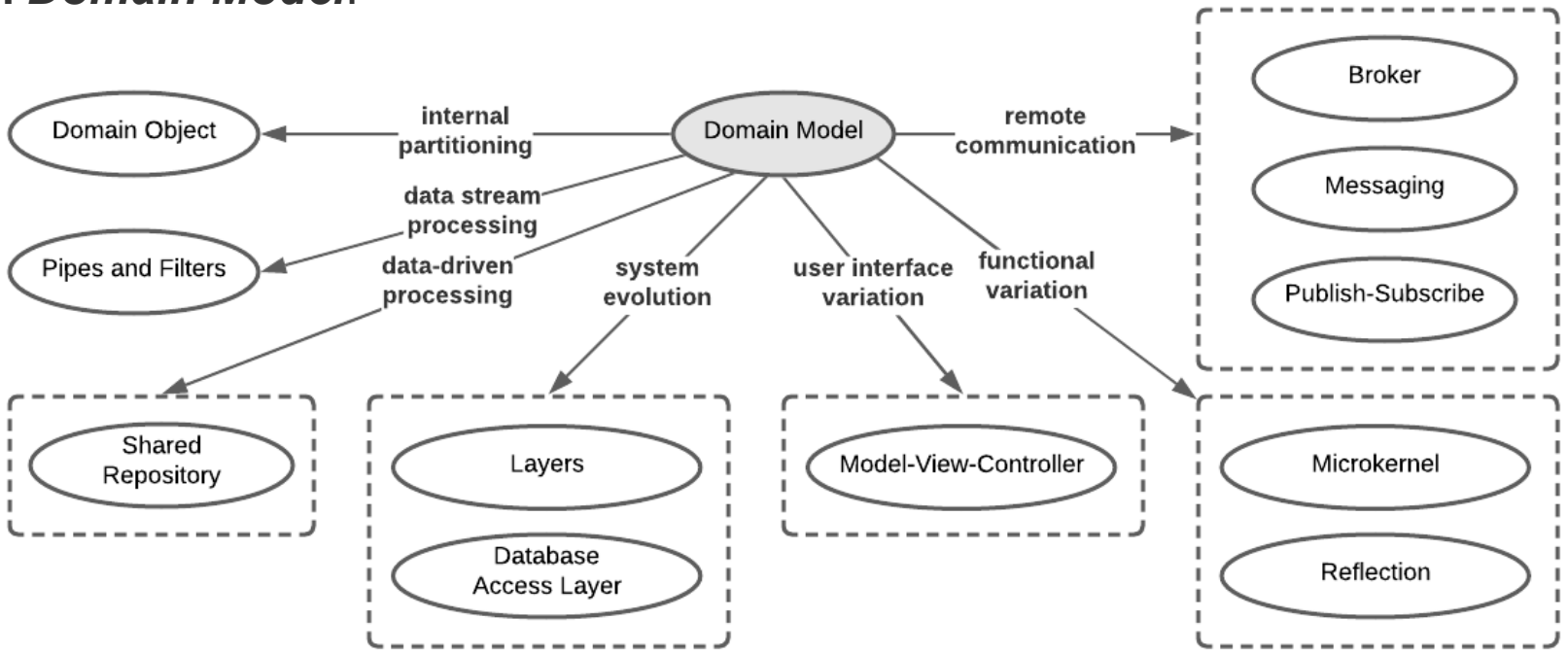
However, **we will not explicitly discuss all patterns that are referenced in this deck**. Feel free though to **study them yourself in future**.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Domain Model

An a**rchitecture pattern language excerpt representing patterns associated with *Domain Model***.

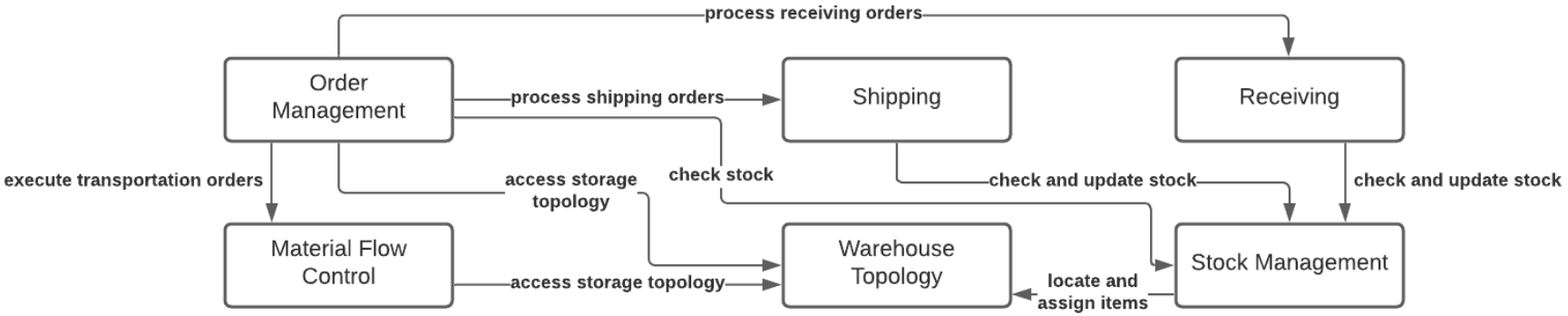


From Mud to Structure
Domain Model

A *Domain Model* creates **a model that defines and scopes a system's business responsibilities and their variations**.

Model **elements are abstractions meaningful in the application domain**, while their **roles and interactions reflect the domain workflow**.

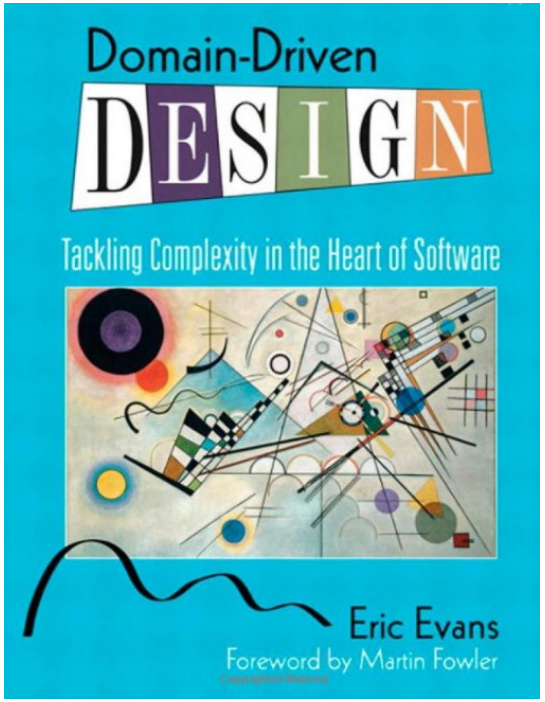
Illustrative example of a simplified *Domain Model* for warehouse management:



From Mud to Structure
Domain Model

A *Domain Model* provides the initial step in transforming requirements into a sustainable software architecture and implementation.

A *Domain Model* is created using an appropriate method, such as **Domain-Driven Design**.



example

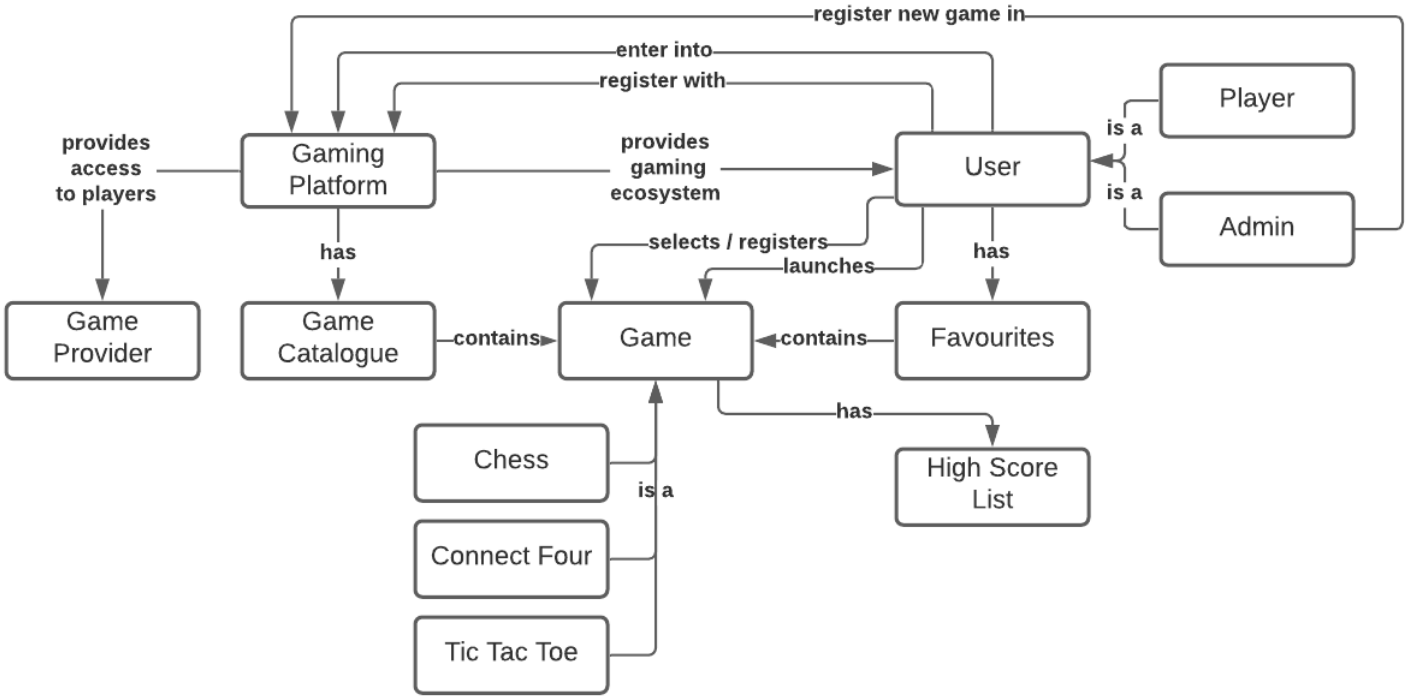
From Mud to Structure
Domain Model

Where have we already created a *Domain Model* during this lecture?

example

From Mud to Structure
Domain Model

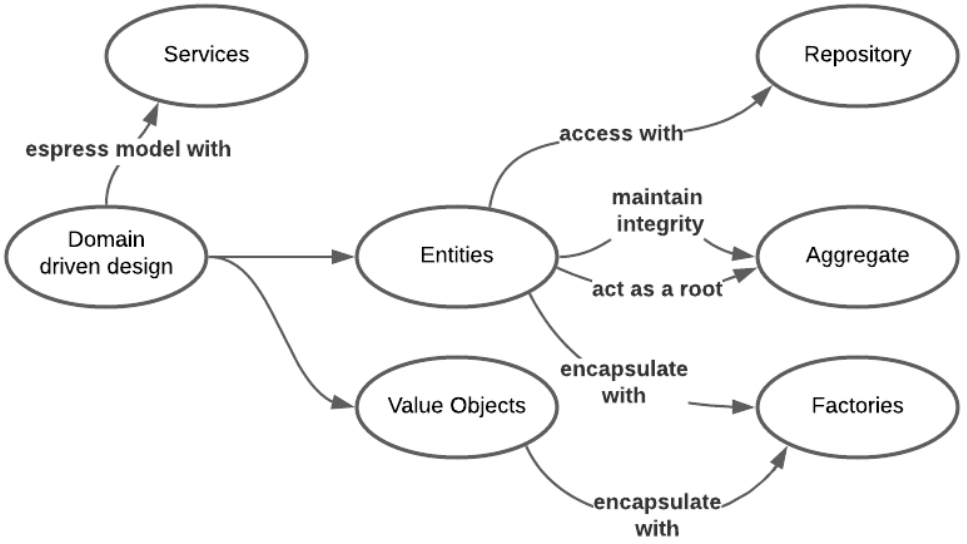
The *Domain Model* we had created for the gaming platform.



From Mud to Structure
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- Entities (reference objects)
- Value objects
- Aggregates
- Associations
- Service objects (services)
- Domain events
- Modules (modules, packages)
- Factories
- Repositories



From Mud to Structure
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- **Entities (reference objects)**

  Objects of the model, which are not defined by their properties, but by their identity. For example, a person is usually represented as an entity. Thus, a person remains the same person if his or her properties change; and he or she is different from another person even if the latter has the same properties. Entities are often modelled using unique identifiers.

- **Value objects**

  Objects of the model that do not have or require a conceptual identity and are thus defined solely by their properties. Value objects are usually modelled as immutable objects, which makes them reusable (cf. Flyweight) and distributable.

- **Aggregates**

  Aggregates are combinations of entities and value objects and their associations to a common transactional unit. Aggregates define exactly one entity as the only access to the entire aggregate. All other entities and value objects may not be statically referenced from outside. This guarantees that all invariants of the aggregate and the individual components of the aggregate can be ensured.

- **Associations**

  Associations, as defined in UML, are relationships between two or more objects of the domain model. Here, not only static relationships defined by references are considered, but also dynamic relationships that arise, for example, only through the processing of SQL queries.

From Mud to Structure
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- **Service objects (services)**

  In domain-driven design, functionalities that represent an important concept of the domain model and conceptually belong to several objects of the domain model are modelled as independent service objects. Service objects are usually stateless and therefore reusable classes without associations, with methods that correspond to the provided functionalities. These methods are passed the value objects and entities necessary to process the functionality.

- **Domain events**

  Domain events are objects that describe complex, possibly dynamically changing, actions of the domain model that cause one or more actions or changes in the domain objects. Domain events also enable the modelling of distributed systems. The individual subsystems communicate exclusively via domain-oriented events, so they are strongly decoupled and the entire system is thus more maintainable and scalable.[9].

- **Modules (modules, packages)**

  Modules divide the domain model into functional (not technical) components. They are characterized by strong internal cohesion and low coupling between the modules.

From Mud to Structure
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- **Factories**

  Factories are used to outsource the creation of domain objects to special factory objects. This is useful if either the creation is complex (and, for example, requires associations that the subject object itself no longer needs) or the specific creation of the subject objects should be able to be exchanged at runtime. Factories are usually implemented by generating design patterns such as abstract factory, factory method or builder.

- **Repositories**

  Repositories abstract the persistence and search of business objects. Repositories separate the technical infrastructure and all access mechanisms to it from the business logic layer. For all business objects that are loaded via the infrastructure layer, a repository class is provided that encapsulates the loading and search technologies used from the outside. The repositories themselves are part of the domain model and thus part of the business logic layer. They are the only ones that access the objects of the infrastructure layer, which are usually implemented using the design patterns Data Access Objects, Query Objects or Metadata Mapping Layers.

From Mud to Structure
Domain Model

Once a *Domain Model* has matured to the point where it adequately portrays the functional responsibilities of an application, as well as their variations, the next step is to **transform the model into a concrete architecture**.

Several patterns help to arrange and connect the elements of a *Domain Model* to support specific styles of computation:

- **Pipes and Filters** is suitable for applications that process data streams.
- **Shared Repository** helps to organize data-driven applications.
- **Layers** groups elements of the domain model that share similar responsibilities, properties, or granularity into separate layers.
- **Model-View-Controller** separate user interface adaptations without the need to change or modify the realisation of business logic.
- **Microkernel** partitions applications into core functionality, version-specific functionality, and version-specific APIS, to support different product variations.
- **Reflection** objectifies specific aspects of a system's structure and behavior to support runtime flexibility in terms of how its functionality executes and can be used by its clients
- **Database Access Layer** decouples application functionality from a relational database to make it easy to replace the database.

From Mud to Structure
Domain Model

Each self-contained and coherent entity or responsibility within the application's baseline is represented as a separate *Domain Object* to provide a defined software realization that addresses its specific functional, operational, and developmental requirements.

In a distributed system, ***Domain Objects* communicate via a distribution system**. For example, via …

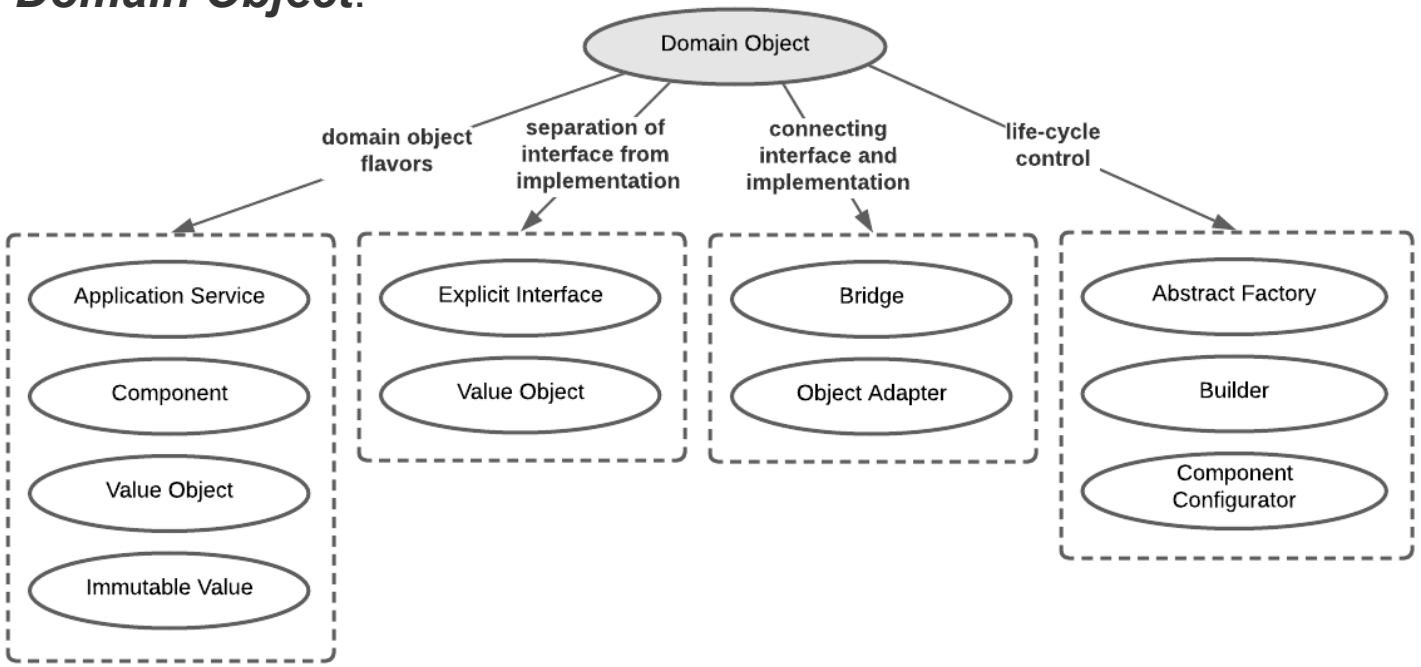
- **Broker** that supports applications whose components communicate via remote method invocation.
- **Messaging** supports systems in which components exchange asynchronous messages.
- **Publish-Subscribe** mediates communication between components that coordinate their processing via notifications of changes to their state.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Domain Object

An a**rchitecture pattern language excerpt representing patterns associated with *Domain Object***.



example

From Mud to Structure
Domain Object

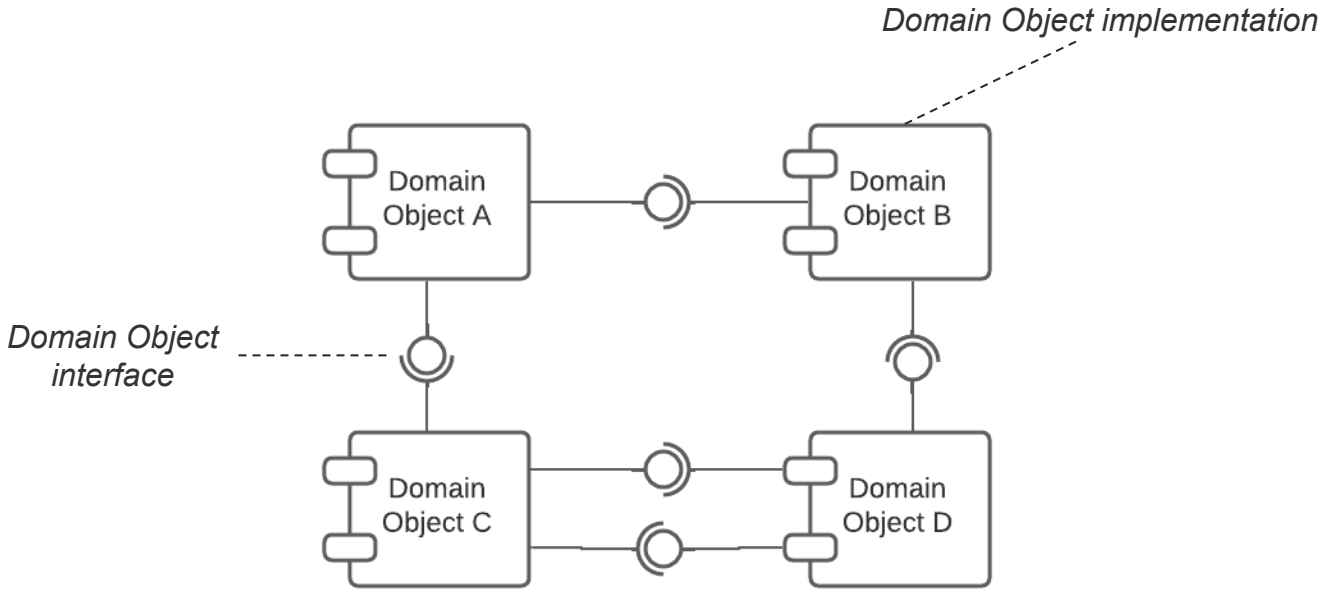
A ***Domain Object*** **encapsulates each distinct functionality of an application in a self-contained building block**.

Which concept we have looked at in-depth does this definition remind you of?

example

From Mud to Structure
Domain Object

An interpretation of the ***Domain Object*** pattern is what we have discussed as Component.



From Mud to Structure
Domain Object

The specific partitioning of a system's responsibilities into *Domain Objects* is based on one or more granularity criteria.

- **Application Service** is a *Domain Object* that encapsulates self-contained and complete business feature or infrastructure aspect of an application, such as banking, flight booking, or logging service.
- **Component** is a *Domain Object* that either encapsulates a functional building block such as an income tax calculation or a currency conversion, or a domain entity such as a bank account or a user.

A *Domain Object* can aggregate other *Domain Objects* of the same or smaller granularity.

Several related patterns support, complement or utilize *Domain Object*:

- **Explicit Interface**. Establish for each *Domain Object* one or more *Explicit Interfaces* and distinguish between imported and exported interfaces.
- Use **Bridge** or **Object Adapter** to decouple the *Explicit Interface* of a *Domain Object* from its implementation, such that the two can vary independently.
- *Domain Objects* are often associated with an **Abstract Factory** or **Builder** that allows clients to obtain access to their Explicit Interface and to manage their liefetime transparently.

example

From Mud to Structure
Domain Object

Can you explain the Adapter pattern?

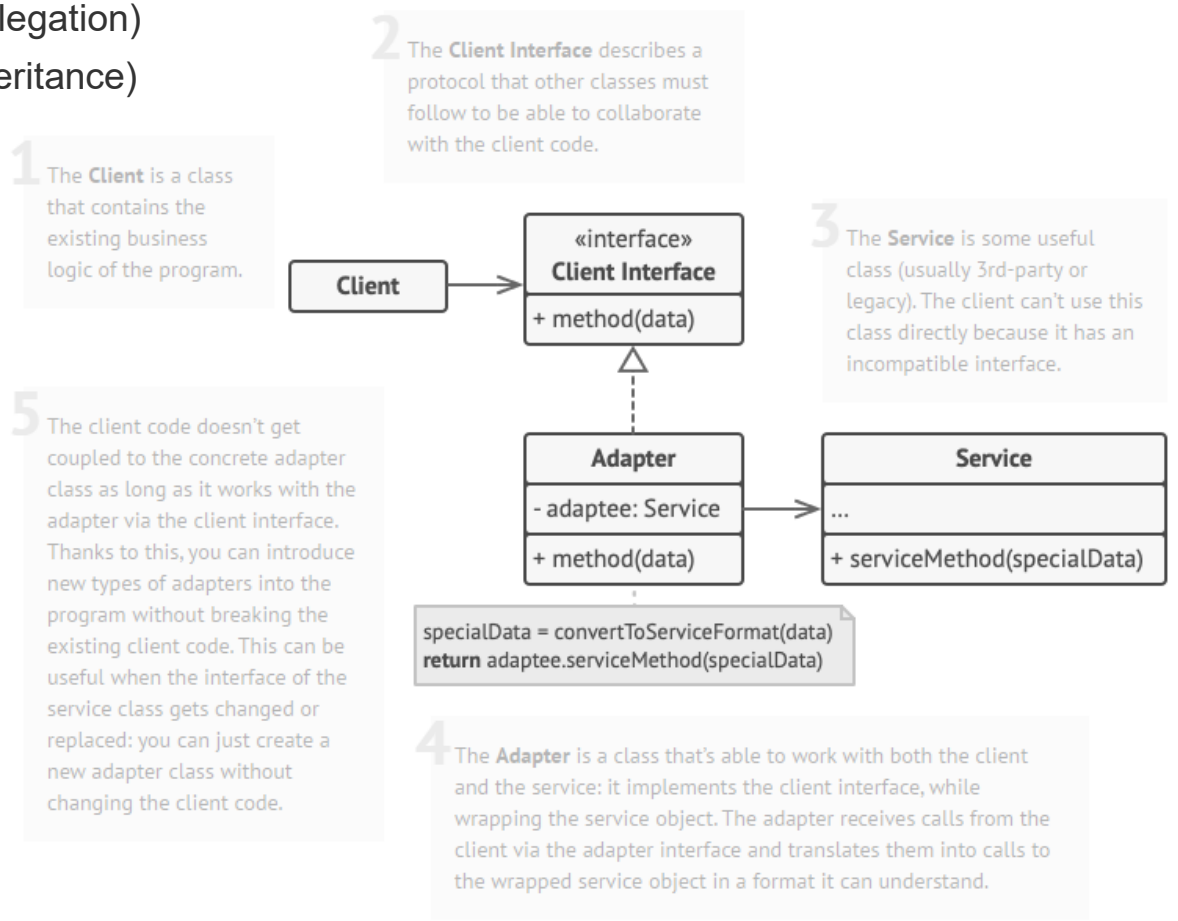
example

From Mud to Structure
Domain Object

Two flavours of the *Adapter* pattern exist:

- Object Adapter (delegation)
- Class Adapter (inheritance)

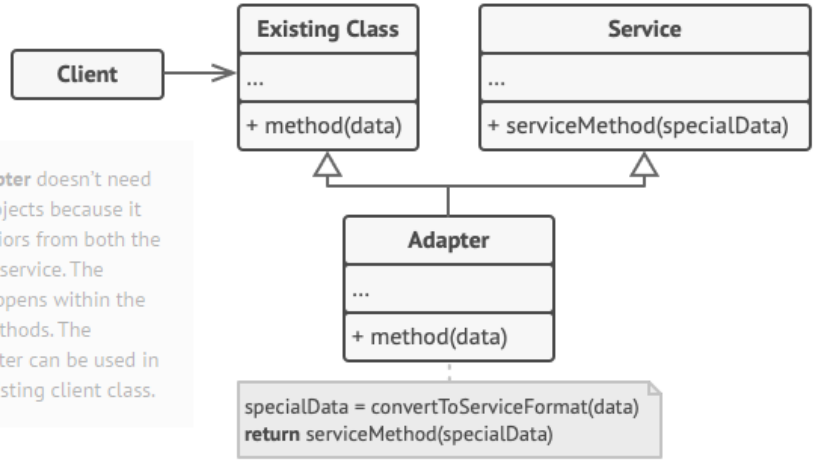
Object Adapter



example

From Mud to Structure
Domain Object

Class Adapter

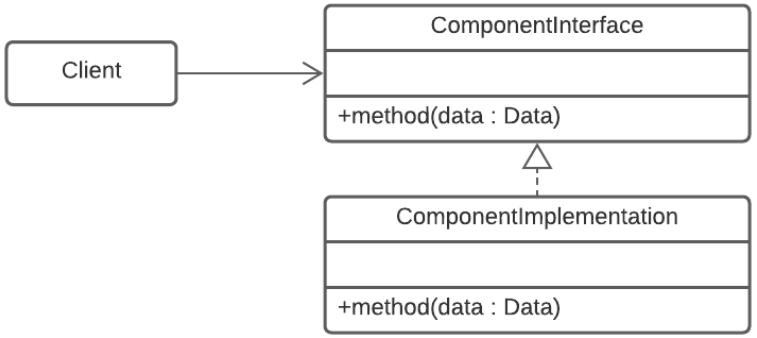


1 The **Class Adapter** doesn't need to wrap any objects because it inherits behaviors from both the client and the service. The adaptation happens within the overridden methods. The resulting adapter can be used in place of an existing client class.

example

From Mud to Structure
Domain Object

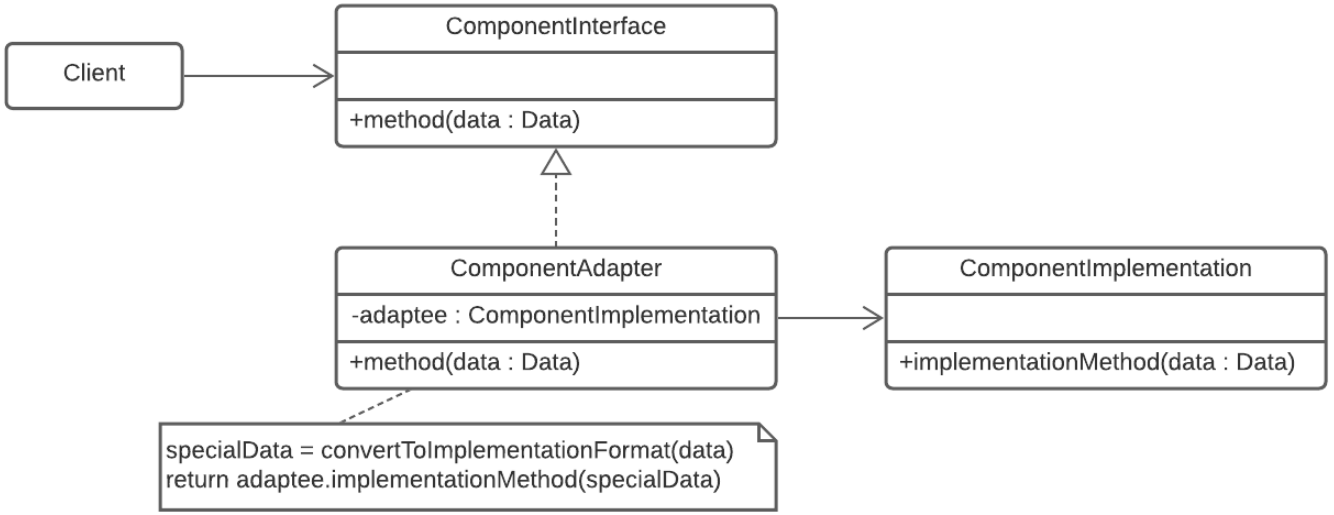
How can we use the **Object Adapter** to connect the *Explicit Interface* of a *Component* to its implementation such that implementing Component and Explicit Interface can evolve more independently?



example

From Mud to Structure
Domain Object

Using an **Object Adapter** to connect the *Explicit Interface* of a *Component* to its implementation.

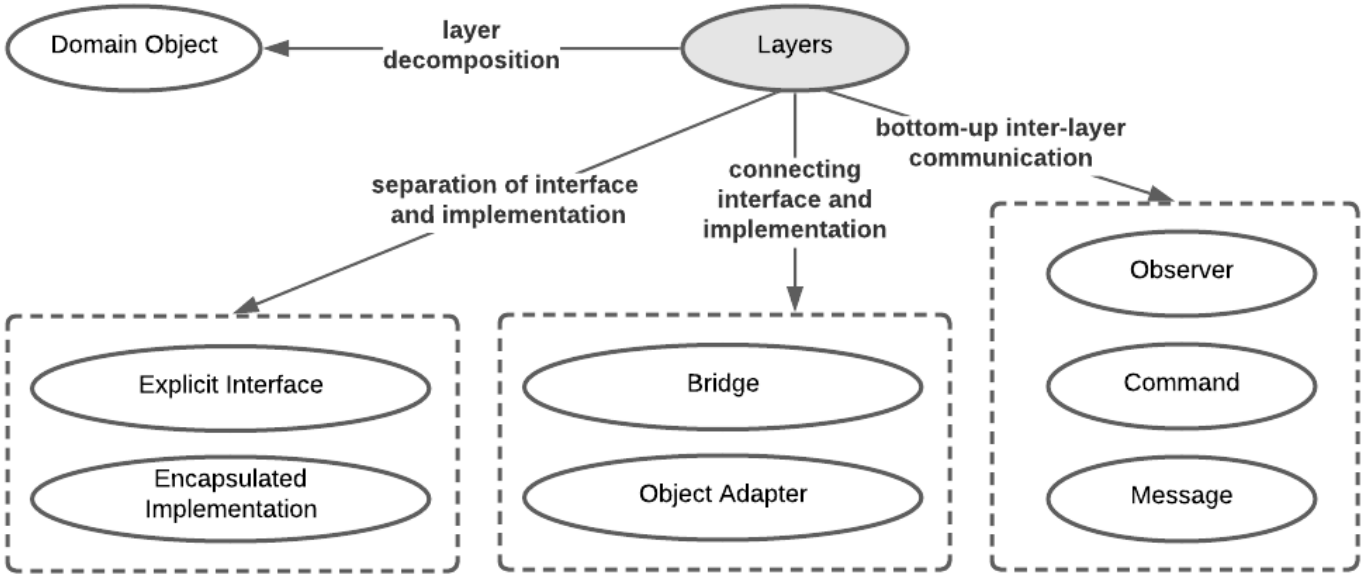


Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Layers

An a**rchitecture pattern language excerpt representing patterns associated with *Layers***.



From Mud to Structure
Layers

The probably **most common scheme to structure software are *Layers* or tiers**.

Components are composed into larger building blocks (i.e., layers). Component composition is based on coherence criteria and component responsibility.

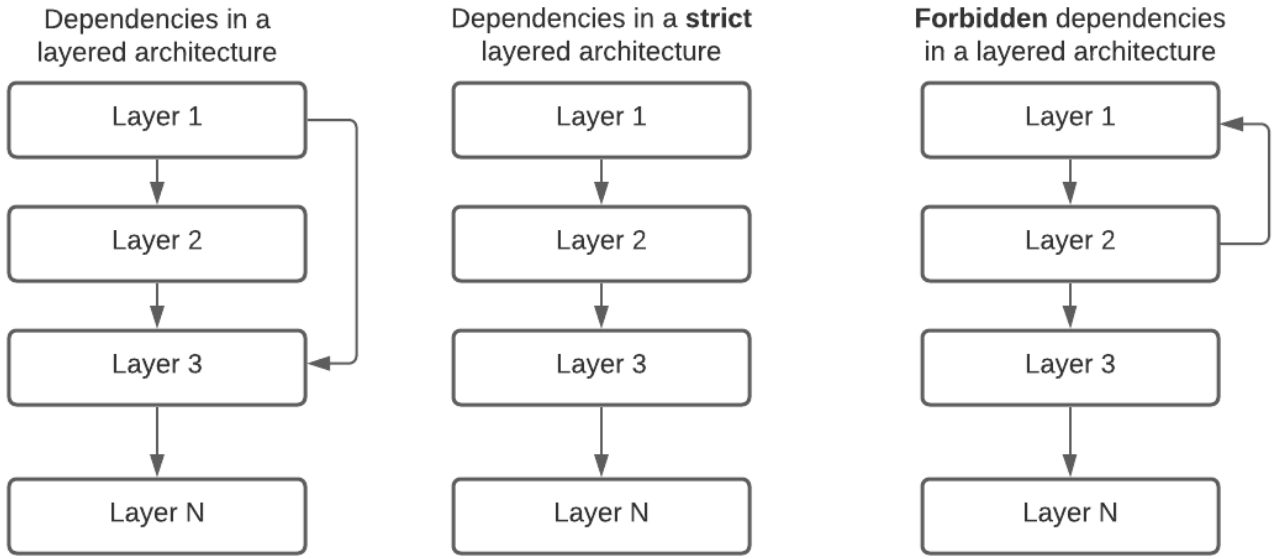
Imagine *layers* as building blocks stacked upon each other. **Components from a layer may only know components from subordinate layers**, but never vice versa.

Within a layer, however, dependencies are not restricted.

Between the layers there are no mutual or cyclic dependencies.

From Mud to Structure
Layers

Strict and less strict *Layer* architectures are differentiated.



From Mud to Structure
Layers

The **arrows on the previous slide are usually omitted**. What is usually meant is that **upper layers depend on lower layers**, but not vice versa.

The term dependency refers to the development view. This means that it refers to import relations between classes and components – not between objects.

Advantages of **strict versus less strict layered architectures** …

- Layers further down can be changed or even completely replaced without affecting layers further up.
- The lower dependencies also improve the situation during testing.

Advantages of **less strict versus strict layer architectures** …

- **Less strict**: more flexible planning of responsibilities of individual layers, since one does not have to plan ahead early which capabilities of a layer are needed "much higher up".
- **Strict**: you usually also save code for "just passing it through", or you can shorten access paths and thus increase performance, for example.

The **trade-off between strict and less strict layer architectures is therefore mostly between performance and modifiability**.

From Mud to Structure
Layers

The **criteria along which partitioning into *Layers* occurs can be defined along various dimensions**, such as …

- **Abstraction**. Partitioning a system into presentation, logic, and data layers.
- **Granularity**. Partitioning a system into business process and business object layers.
- **Hardware distance**. Partitioning a system into operating system, abstraction, communication protocol, and application layers.
- **Rate of change**. Partitioning along the rate of change criteria separates functionalities that evolve independently of one another.

In most systems multiple dimensions are combined. For example, decomposing a system into presentation, logic, and data layers is a layering according to both levels of abstraction and rate of change.

From Mud to Structure
Layers

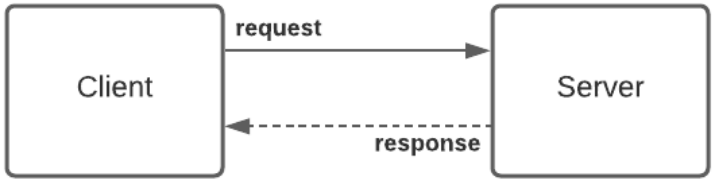
Layered architectures are common both in business systems and in systems used as platforms for business systems.

The **layers are sometimes also referred to as tiers**.

Simple **client-server architectures already represent two tiers**. In 2-tier architectures, the client knows the server, but not vice versa.

The server is ready for requests from the client. The client connects to the server and sends a request.

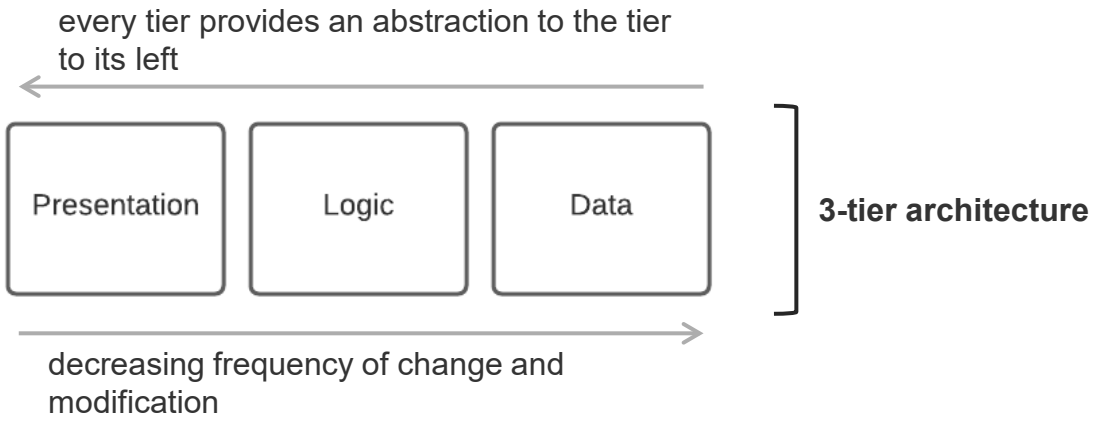
The server responds to the client's request. Servers are often multithreaded in order to be able to efficiently answer simultaneous client requests.



From Mud to Structure
Layers

3-tier and 4-tier architectures are particularly common.

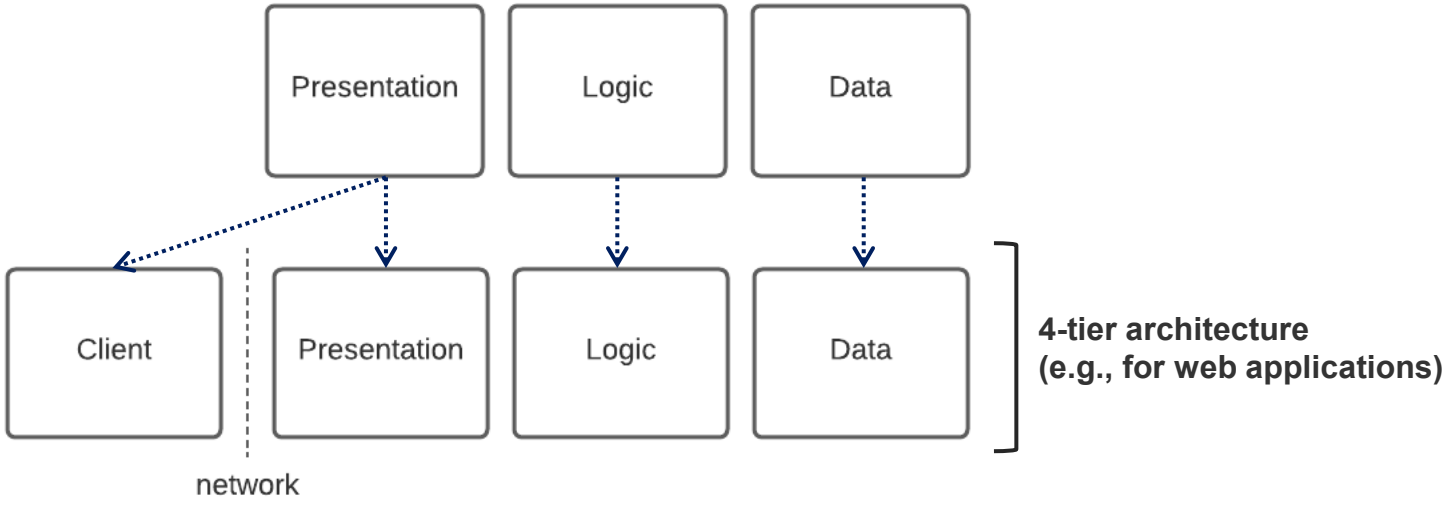
The **basic principle in such architectures** is the separation between the provision and persistent storage of **data**, the **logic** for systematic data manipulation, and the **presentation** of data via user interfaces.



From Mud to Structure
Layers

3-tier and 4-tier architectures are particularly common.

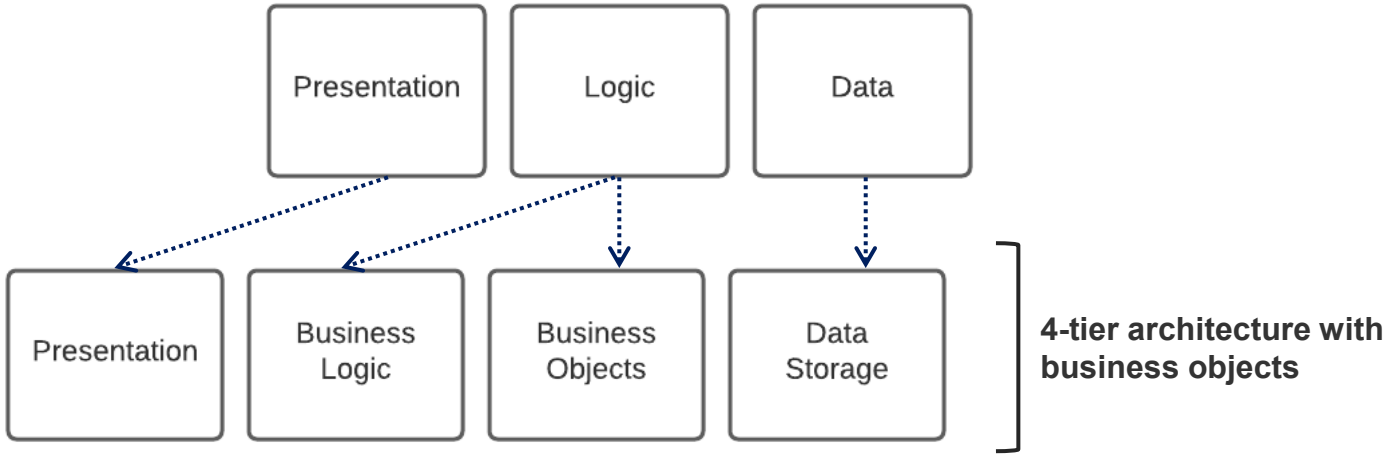
In **web applications, the presentation layer is divided into a client and a server part** due to the their inherent physical distribution. Note, that the application-specific code in the client tier is obtained from the server tier.



From Mud to Structure
Layers

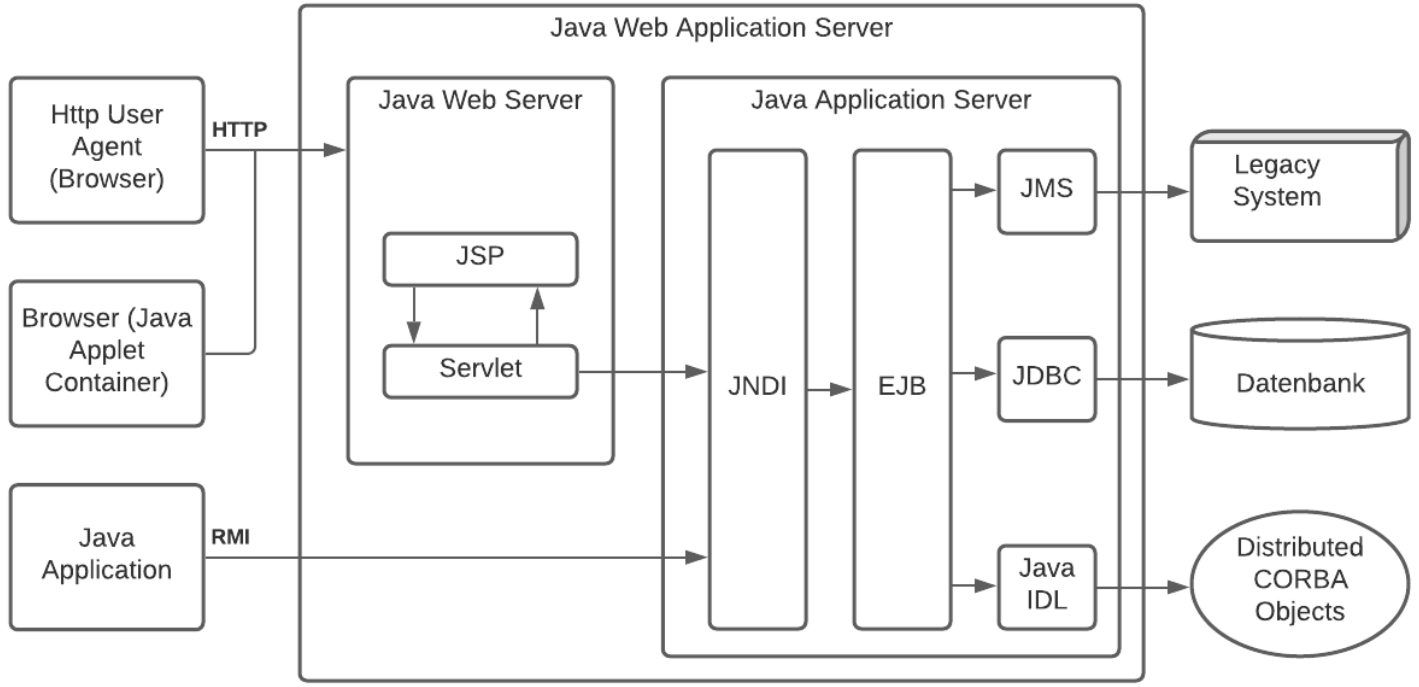
3-tier and 4-tier architectures are particularly common.

Sometimes the **logic layer is also divided into an object-oriented business object layer and a database layer** implemented mostly classically with relational models.



From Mud to Structure
Layers

The **Java platform (Java EE) supports this model by a corresponding programming and server model** as well as suitable Java API standards.



From Mud to Structure
Layers

In layered architectures, **dependencies must be considered at layer boundaries**.

As we have already seen with components, there are often **constraints on dependencies in the development view that conflict with the necessary relationships in the logical view** (i.e., relationships between objects at runtime).

If A and B represent arbitrary classes or interfaces, then **"A is dependent on B" in the development view means that B must be present at compilation time of A**.

For example, because B is used as type of variables or parameters in A or because methods declared in B are used by A (even if this is "only" a constructor).

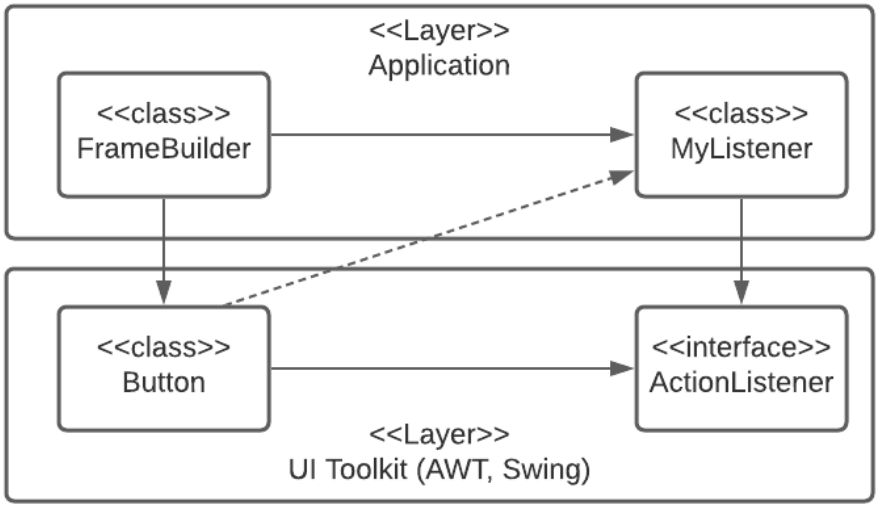
Such dependencies should exist if possible only to classes, which are inherently close.

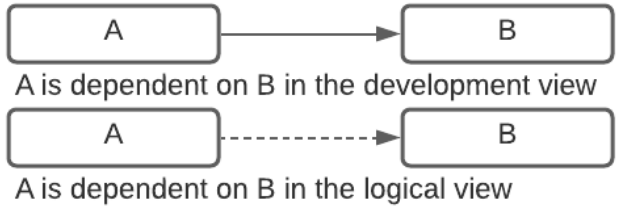
However, calls between objects are frequently necessary also against the layer structure.

example

From Mud to Structure
Layers

The contradiction can be resolved by designs, as suggested by the Observer Pattern and as you know it from common UI toolkits.






```
public class FrameBuilder extends JFrame {
       public FrameBuilder() {
            setSize(400, 300);

             JButton button = new JButton("Click Me");
             button.addActionListener(new MyListener());
             add(button);

             setDefaultCloseOperation(EXIT_ON_CLOSE);
       }
       public static void main(String[] args) {
             FrameBuilder demo = new FrameBuilder();
             demo.setVisible(true);
        }
}
```

```
public class MyListener implements ActionListener {
       public void actionPerformed(ActionEvent e) {
            System.out.println("Button was clicked");
       }
}
```

```
Package com.fhnw.exercise.architecturepatterns.observer;
```

From Mud to Structure
Layers

Several related patterns support, complement or utilize *Layers*:

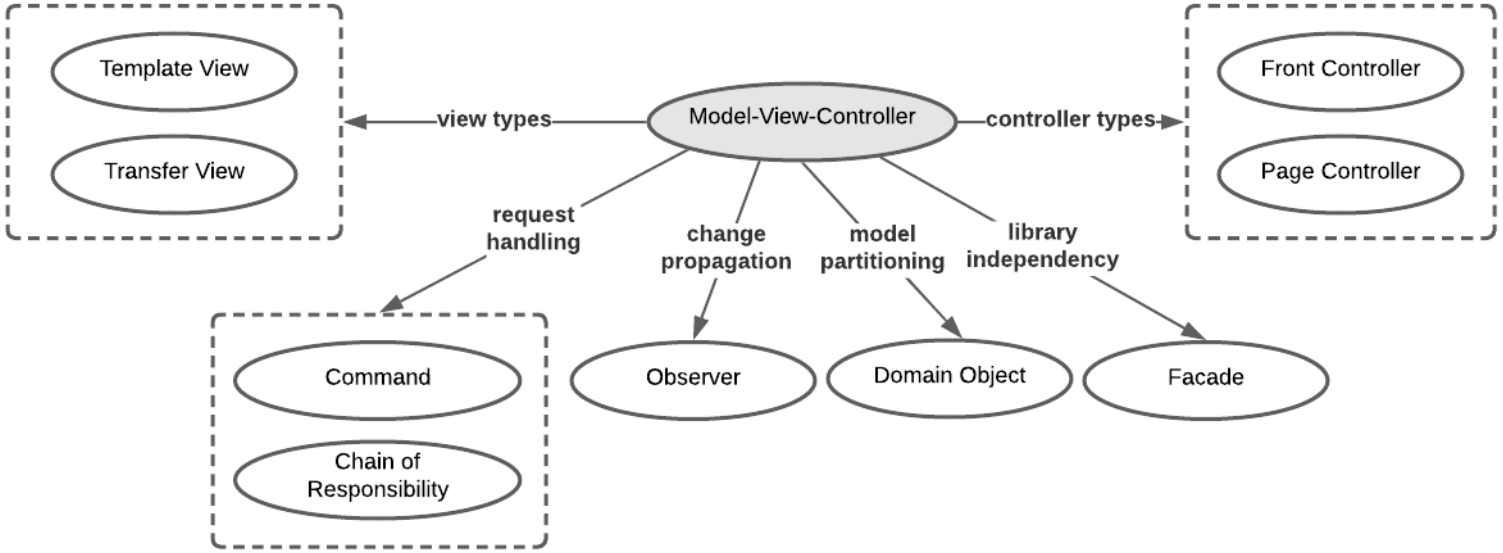
- **Domain Object.** Typically, each self-contained and coherent responsibility within a layer is realized as a separate *Domain Object* (also see: Component), to further partition the layer into tangible parts that can be developed and evolved independently.
- **Explicit Interface**. Split each layer into an *Explicit Interface* that publishes the interfaces of those domain objects whose functionality should be accessible by other layers.
- **Observer**. Control flow that originates from the 'bottom' of a stack (e.g., network communication stack) is often triggered via an *Observer*-based call-back infrastructure.
- **Command**. Lower layers can pass service requests to higher layers via notifications realized as *Commands*.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Model-View-Controller

An a**rchitecture pattern language excerpt representing patterns associated with *Model-View-Controller***.



From Mud to Structure
Model-View-Controller

Model-View-Controller (MVC) is another very common architectural pattern.

It was designed in its original form in 1979 for the **Smalltalk language and programming environment**. At that time the development of the hardware from ASCII terminals to bitmap displays and pointing devices made more complex user interactions possible for the first time.

The **need arose to clearly distinguish between data processing and the machinery for user interaction**, among other things, in order to be able to adapt user interfaces to new developments without affecting the calculations for the business processes.

example

From Mud to Structure
Model-View-Controller

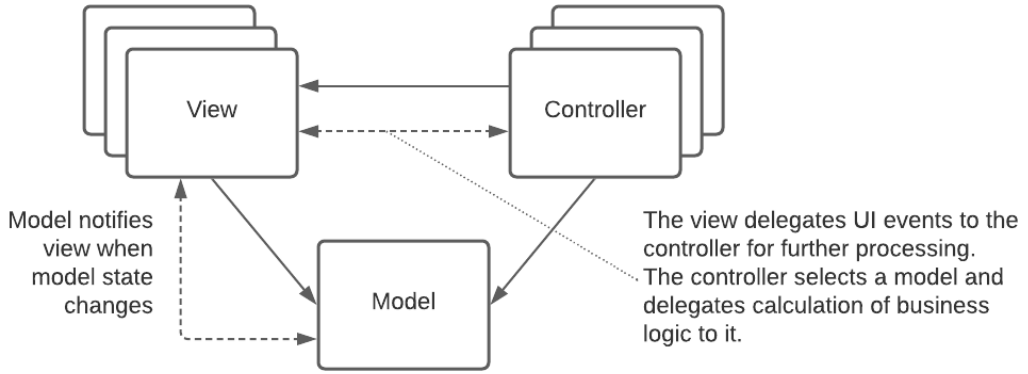
What is the key idea of the MVC pattern, what are its components and how are these interrelated?

example

From Mud to Structure
Model-View-Controller

The idea of the MVC pattern is that, in addition to a **business data and logic layer (model)**, there is an **user interface layer (view)** and a **layer that mediates between them (controller)**.

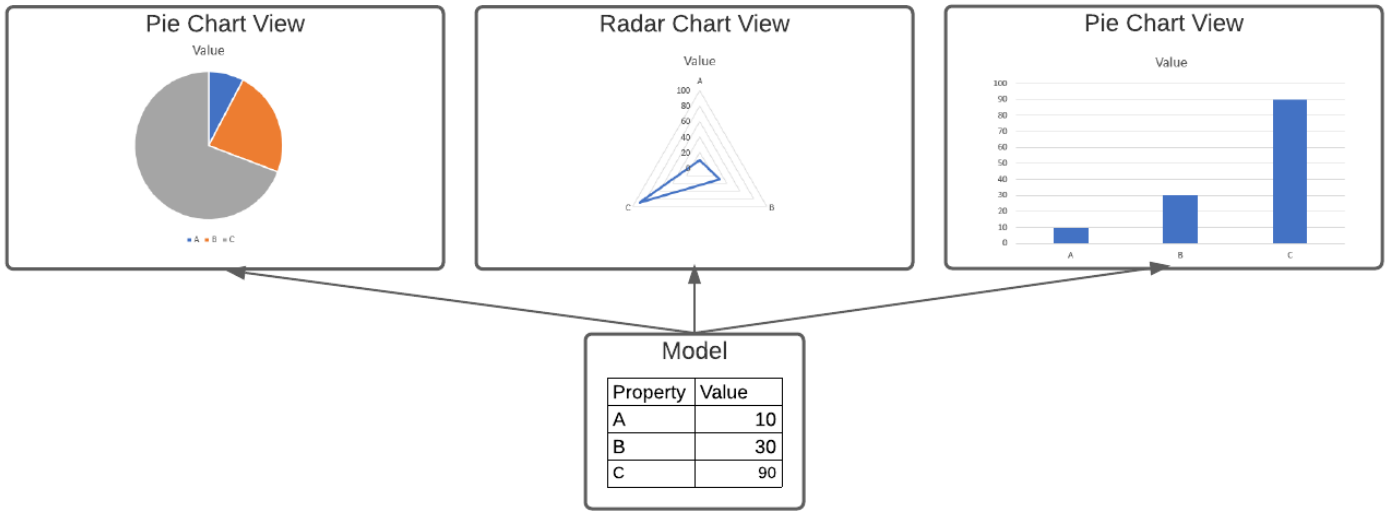
Mediating means that the controller selects business logic in response to an event from a view and selects a view suitable to present a calculated business logic result.



From Mud to Structure
Model-View-Controller

The **primary goals of the MVC pattern**, in addition to clearly separating responsibilities, are:

1. to enable development of alternative views and controllers without requiring business logic to be changed
2. to allow multiple views per model at the same time
3. enable similar-looking views with different behavior for user actions
4. allow easy to automate tests on the business logic

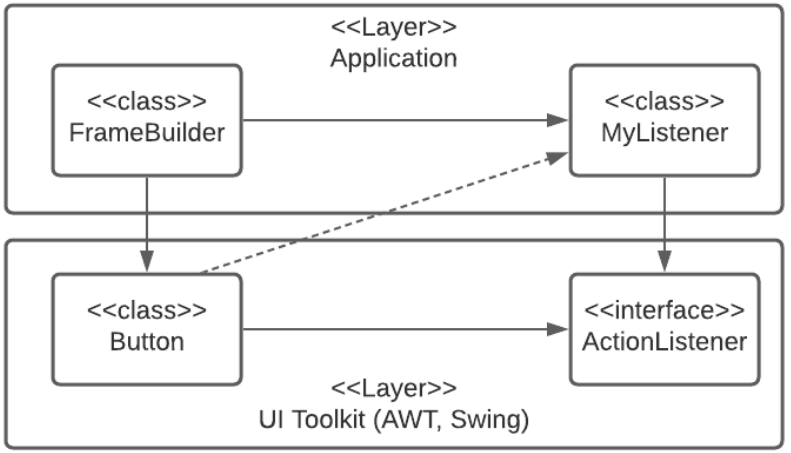


From Mud to Structure
Model-View-Controller

To achieve the MVC goals, the classes that make up the **model must not depend on those of the view**.

However, **the view must be able to represent changes that occur in the model data**. Changes to the model data must therefore be able to cause changes in the representation.

That is why the **observer pattern is an integral part of MVC**. **Views are registered with the model as Observer** and are then informed by the model about changes.



From Mud to Structure
Model-View-Controller

To **relieve programmers from recurrently implementing the MVC pattern in their designs, many frameworks and toolkits provide basic mechanisms for MVC support** – mechanisms which also include an adapted observer pattern.

Developers can thus concentrate on application-specific configurations and are freed from having to manage the MVC pattern for all their views, controllers and models, themselves.

From Mud to Structure
Model-View-Controller

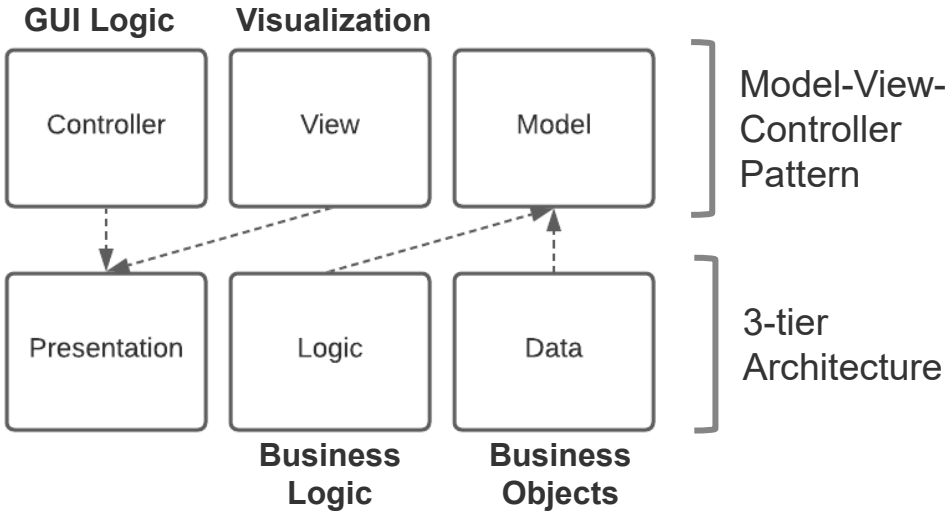
The **distinction between GUI logic and business logic is not to be neglected**. The latter means the logic behind the manipulation of (persistent) data, i.e. the flow control of (complex) business processes.

While the GUI logic is part of the responsibility of the user interface the business processes are subject to other changes and other quality measures. Typically other core competencies are required for programming than for GUI design. Therefore, you want to leave the business logic untouched even if the GUI changes.

Conversely, of course, **the model must not be made dependent on implementation details of the presentation**, which is why the GUI logic does not belong in the model.

From Mud to Structure
Model-View-Controller

If you want to **separate business logic from data management, this actually means that you divide the model itself into two layers** again.



From Mud to Structure
Model-View-Controller

Two aspects in particular are **characteristic of the MVC pattern**:

- The GUI logic depends on the view - not vice versa
- View is informed about changes by the model (observer pattern)

If these two characteristics are not present, then strictly speaking we should no longer speak of an MVC pattern but at best an MVC derivate.

In the case of web applications, there is a direct conflict between the REST model, which is useful for this purpose, and the MVC pattern.

RESTful services are characterized, among other things, by the fact that the client sends a request to the server and that the server responds once. After this request-response cycle there is no connection maintained between the client and the server.

This means that notifications about state changes from the server to the client is not possible.

example

From Mud to Structure
Model-View-Controller

How is the MVC pattern applied in the context of web applications?

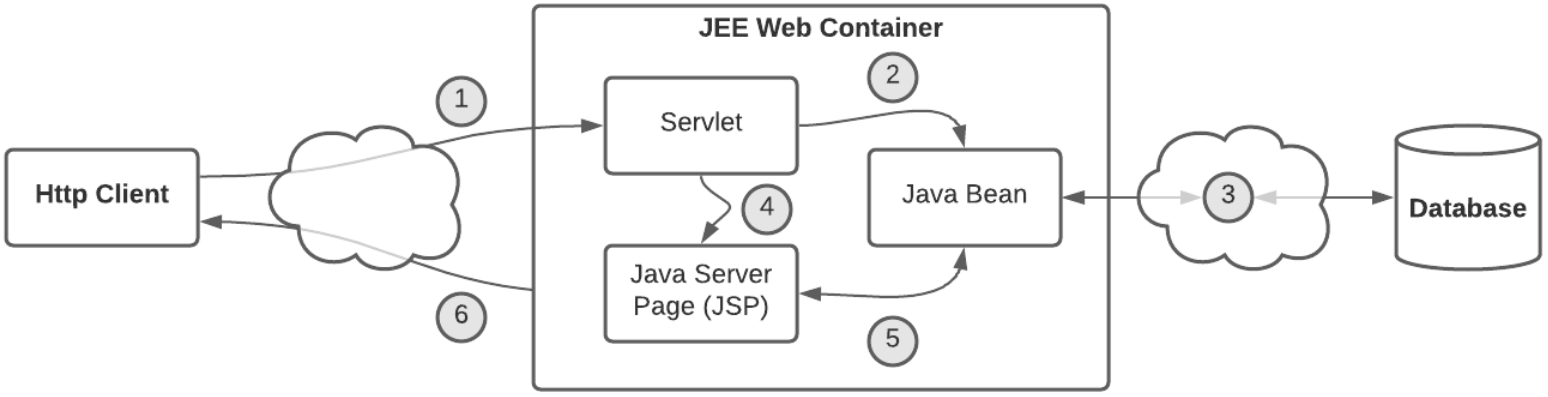
example

From Mud to Structure
Model-View-Controller

In the case of web applications the MVC pattern can therefore be used at best on the server.

The actually displayed view is only updated if the client asks for it.

In the context of Java Web technology based on servlets and JSP, this is why we speak of the MVC2 pattern – a slight modification of the MVC pattern.



example

From Mud to Structure
Model-View-Controller

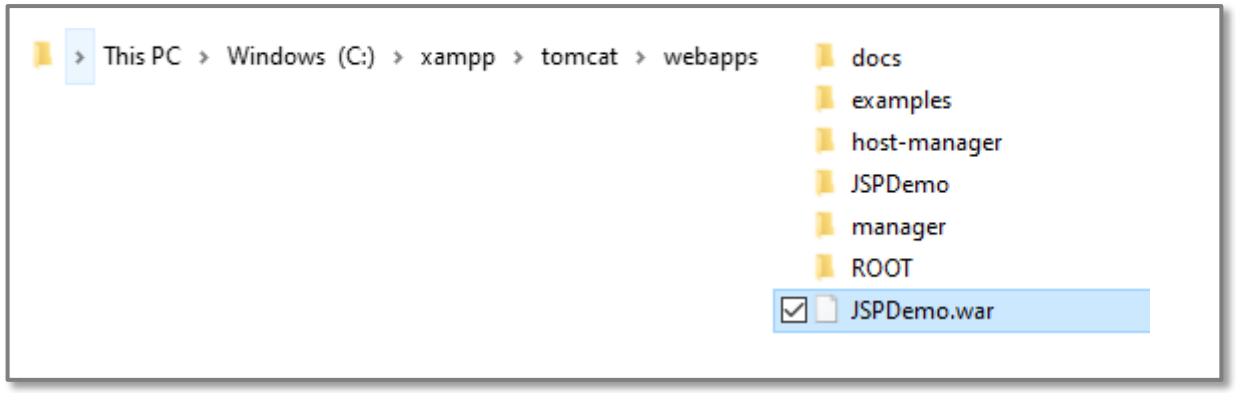
Web application dynamics:

1. An Http request is sent from the Http client to the Http server (JEE Web Container) and received there by the servlet.
2. The servlet acts as a controller. In this role, it selects the domain-oriented component (model) that is responsible for the calculation (i.e., JavaBean).
3. The business component (Model) implements business logic and accesses e.g. a database for it.
4. After the calculation is done, the servlet forwards the control (including the calculated result) to the Java Server Page (View) (i.e., request dispatching).
5. The Java Server Page is responsible for the html output, for which it uses the model data from the JavaBean.
6. The result of the Java Server Page is sent back to the browser as Http Response.

example

From Mud to Structure
Model-View-Controller

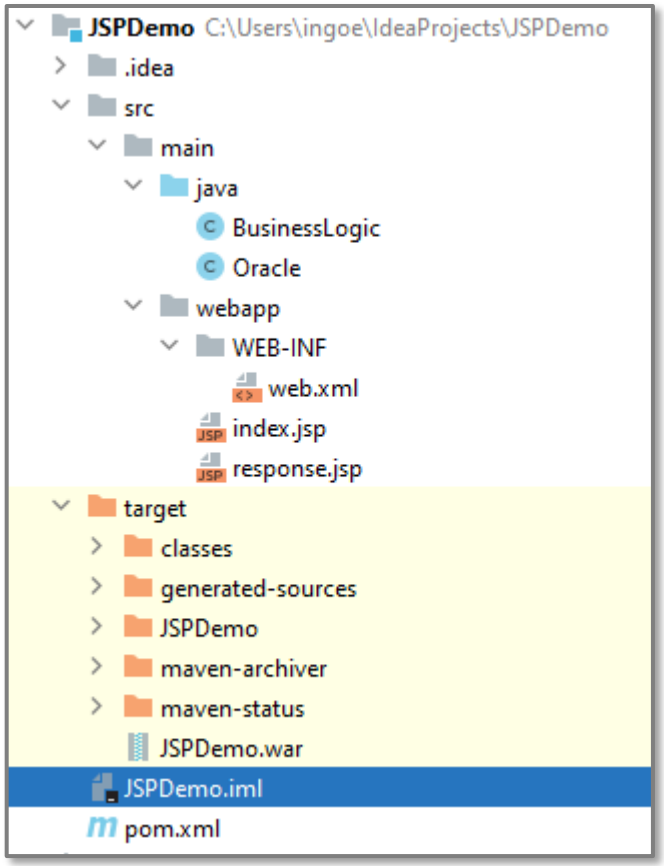
Tomcat: *webapps* directory



example

From Mud to Structure
Model-View-Controller

IntelliJ: web application configuration



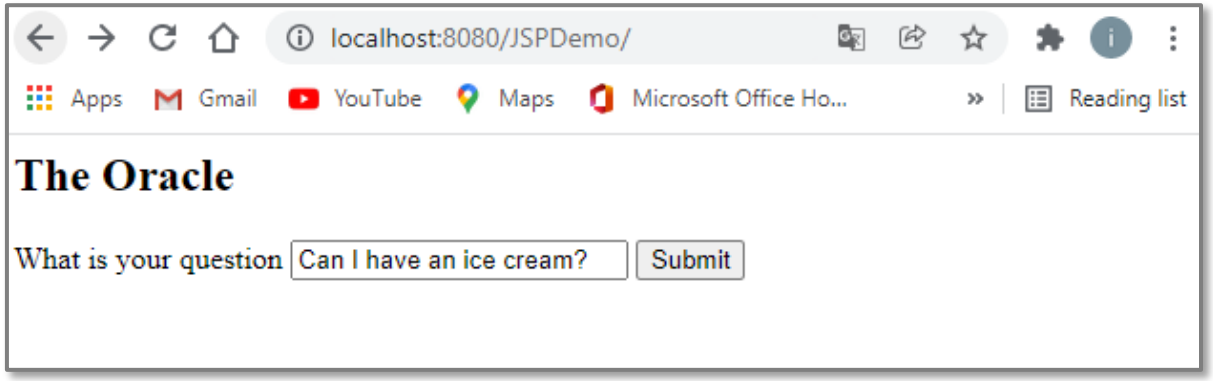
example

From Mud to Structure
Model-View-Controller

IntelliJ: index.jsp

```
<html>
<body>
<h2>The Oracle</h2>
<form action="queryTheOracle" method="POST">
    <span>What is your question</span>
    <input type="text" name="question">
    <input type="submit" name="submit">
</form>
</body>
</html>
```

Http client: rendered index.jsp



example

From Mud to Structure
Model-View-Controller

IntelliJ: web.xml (`/queryTheOracle` → `OracleServlet` → `Oracle(.class)`)

```
<web-app>
  <display-name>Archetype Created Web Application</display-name>

  <servlet>
    <servlet-name>OracleServlet</servlet-name>
    <servlet-class>Oracle</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>OracleServlet</servlet-name>
    <url-pattern>/queryTheOracle</url-pattern>
  </servlet-mapping>
</web-app>
```

example

From Mud to Structure
Model-View-Controller

IntelliJ: Oracle.java (*controller*)

```java
public class Oracle extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        // read parameter from post request
        String question = req.getParameter( s: "question");

        // delegate calculation of business logic to business object (POJO)
        String response = new BusinessLogic().calculateResponse(question);

        // store question and calculated response in http request object
        req.setAttribute( s: "question", question);
        req.setAttribute( s: "response",response);

        // delegate presentation to response.jsp (view)
        RequestDispatcher reqDispatcher = req.getRequestDispatcher( s: "/response.jsp");
        reqDispatcher.forward(req,resp);
    }
}
```

example

From Mud to Structure
Model-View-Controller

IntelliJ: BusinessLogic.java (*model*)

```java
public class BusinessLogic {
    public String calculateResponse(String question) {
        // calculate an answer in response to the question
        String result = "No, never ever!";
        return result;
    }
}
```

example

From Mud to Structure
Model-View-Controller

IntelliJ: response.jsp (*view*)

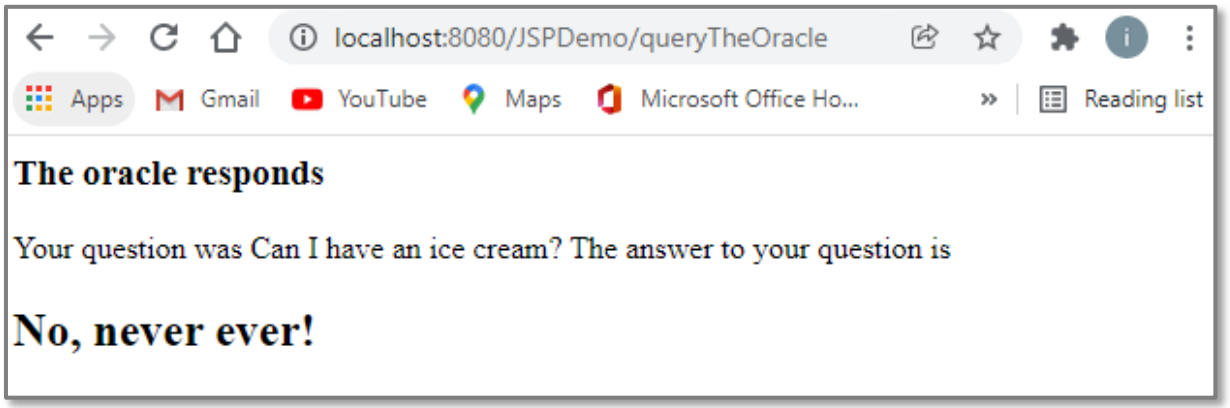
```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page isELIgnored="false" %>
<html>
<head><title>The oracle responds</title></head>
<body>
    <h3>The oracle responds</h3>
    <span>Your question was ${question} </span>

    <span>The answer to your question is </span>
    <h2>
        ${response}
    </h2>
</body>
</html>
```

example

From Mud to Structure
Model-View-Controller

Http client: rendered response.jsp



From Mud to Structure
Model-View-Controller

Several related patterns support, complement or utilize *Model-View-Controller*:

- **Domain Object.** Often the model is partitioned into one or more application *Domain Objects*, one for each self-contained responsibility.
- **Template View**. *Template View* renders model information into a predefined output format (e.g., JSP).
- **Transformation View**. A *Transform View* creates its output by rendering each data element that it retrieves from the model, individually (e.g., XSLT processors).
- **Front Controller**. A *Front Controller* handles all requests on the model. A controller per each model function is most suitable if the model supports a wide range of functions.
- **Page Controller**. A *Page Controller* is appropriate for form-based or page-based user interfaces in which each form or page offers a set of related functions (e.g., a servlet that processes post requests from a specific HTML form qualifies as a page controller).
- **Command**. The requests issued by controllers may be encapsulated into *Command* objects that are passed to associated models (e.g., *Domain Object*). Such a design allows controllers to change transparently to both the views and the model. In addition, it supports the treatment of requests as first class citizen objects. This enables a system to offer house-keeping services like undo/redo or request scheduling.

From Mud to Structure
Model-View-Controller

Several related patterns support, complement or utilize *Model-View-Controller*:

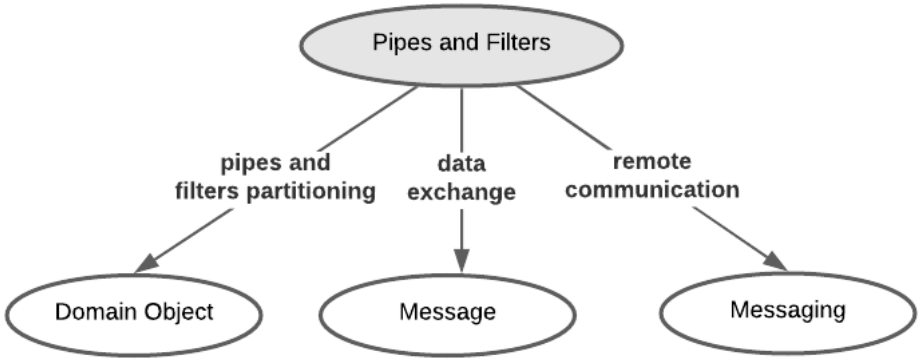
- **Chain of Responsibility.** A *Chain of Responsibility* that connects controllers (e.g., servlets in a web application) simplifies the dispatching of the 'right' controller in response to a specific input. An example of this application of *Chain of Responsibility* is *servlet filtering* in web applications.
- **Façade**. A *Façade* for accessing low-level libraries (e.g., database libraries, graphical libraries for rich user interface applications) enables the views, controllers, and models to be kept independent of a system's platform and supports system portability.
- **Observer**. To support efficient collaboration between model, views, and controllers without breaking the model's independence of user interface aspects, connect them via an *Observer* arrangement. The model is a subject, while the views and controllers are its observers. When the model changes its state, it notifies all registered views and controllers, which in turn update their own state by retrieving the corresponding data from the model.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Pipes and Filters

An a**rchitecture pattern language excerpt representing patterns associated with *Pipes and Filters***.

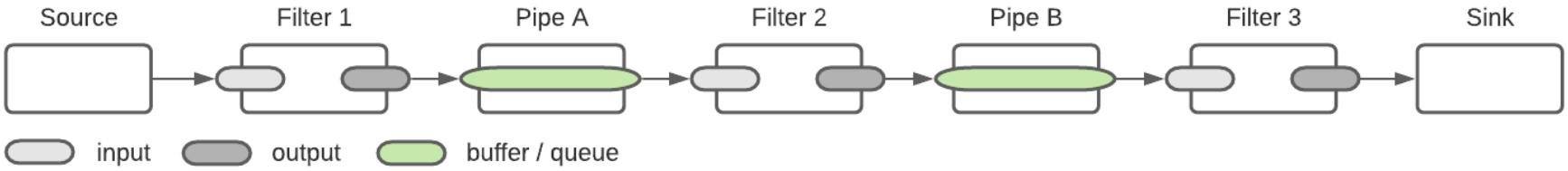


From Mud to Structure
Pipes and Filters

Some applications **process streams of data, where input data streams are transformed stepwise into output data streams**.

However, using common request-response semantics for structuring such types of application is often impractical. Instead an appropriate data flow model must be specified.

The ***Pipes and Filters*** **pattern suggests to divide an application's overall task into several self-contained data processing steps** (i.e., *filters*) and **connect these steps to a data processing pipeline via intermediate data buffers** (i.e., *pipes*).



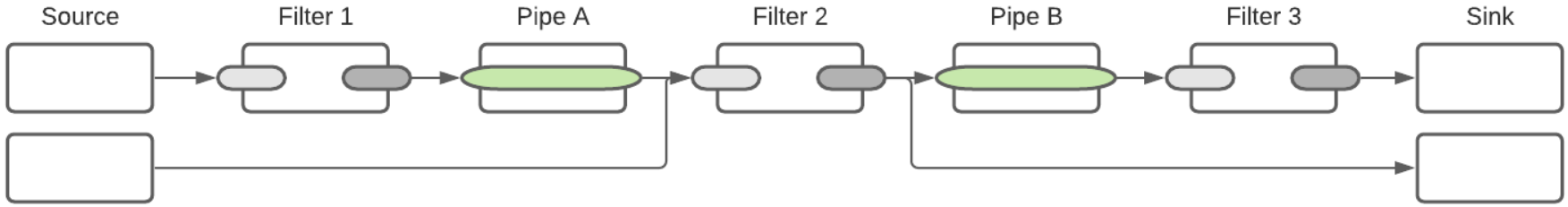
From Mud to Structure
Pipes and Filters

Implement each processing step as a separate filter component that consumes and delivers data incrementally, and **chain the filters such that they model an application's main data flow**.

In the data processing pipeline, **data that is produced by one filter is consumed by its subsequent filters**.

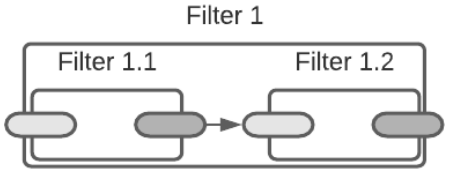
Pipes connect two filters with each other. They are used for **unidirectional data transfer**.

A **filter can have any number of incoming and outgoing pipes**. **Filters can be combined** in many ways and very flexibly. **Multiple sources and/or sinks are conceivable**.



From Mud to Structure
Pipes and Filters

Filters themselves **can be structured recursively**.



The information (i.e., data) flowing from filters to pipes and from there to filters must be understood and processable across the entire processing path. This requires a **standardized message format** that is understood by all building blocks.

Filters are the units of domain-specific computation. Each filter can be implemented as a *Domain Object* that represents a specific, self-contained data processing step.

Pipes are the medium of data exchange and coordination within a Pipes and Filters architecture. Each pipe is a component that implements a policy for buffering and passing data along the filter chain (also see *Messaging*, and *Publish Subscribe*). **Pipes are usually implemented as buffers** (i.e., queues) and can therefore store data until a filter can process it.

From Mud to Structure
Pipes and Filters

While well-designed **filters can work indepdendently and be combined very flexibly in unanticipated contexts**, there is also a **risk of bottlenecks, and single points of failure** that acompanies the *Pipes and Filters* pattern.

Common applications of the *Pipes and Filters* pattern are **compilers** (i.e., scanner, parser, code generator, optimizer), **unix shells** (e.g.: ls -R | grep -i ogg | grep -i soilwork), as well as **integration broker platforms** (e.g., Microsoft BizTalk, Tibco Rendezvous).

example

From Mud to Structure
Pipes and Filters

What is a software design pattern that we have already discussed and applied that is very similar to the *Pipes and Filters* pattern?

example

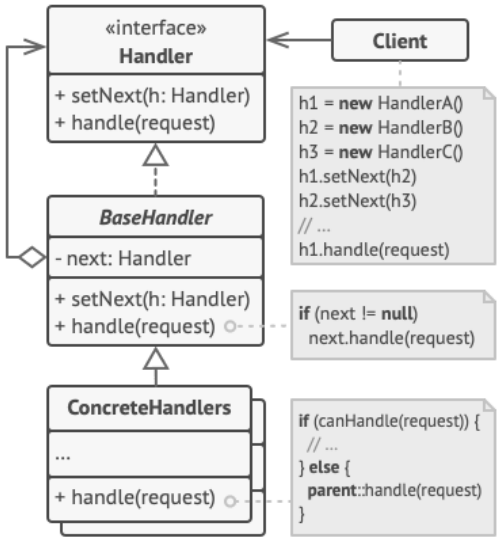
From Mud to Structure
Pipes and Filters

The *Chain of Responsibility* software design pattern that we used as a core pattern to establish a workflow capability for the gaming platform.

1 The **Handler** declares the interface, common for all concrete handlers. It usually contains just a single method for handling requests, but sometimes it may also have another method for setting the next handler on the chain.

2 The **Base Handler** is an optional class where you can put the boilerplate code that's common to all handler classes.

Usually, this class defines a field for storing a reference to the next handler. The clients can build a chain by passing a handler to the constructor or setter of the previous handler. The class may also implement the default handling behavior: it can pass execution to the next handler after checking for its existence.



4 The **Client** may compose chains just once or compose them dynamically, depending on the application's logic. Note that a request can be sent to any handler in the chain—it doesn't have to be the first one.

3 **Concrete Handlers** contain the actual code for processing requests. Upon receiving a request, each handler must decide whether to process it and, additionally, whether to pass it along the chain.

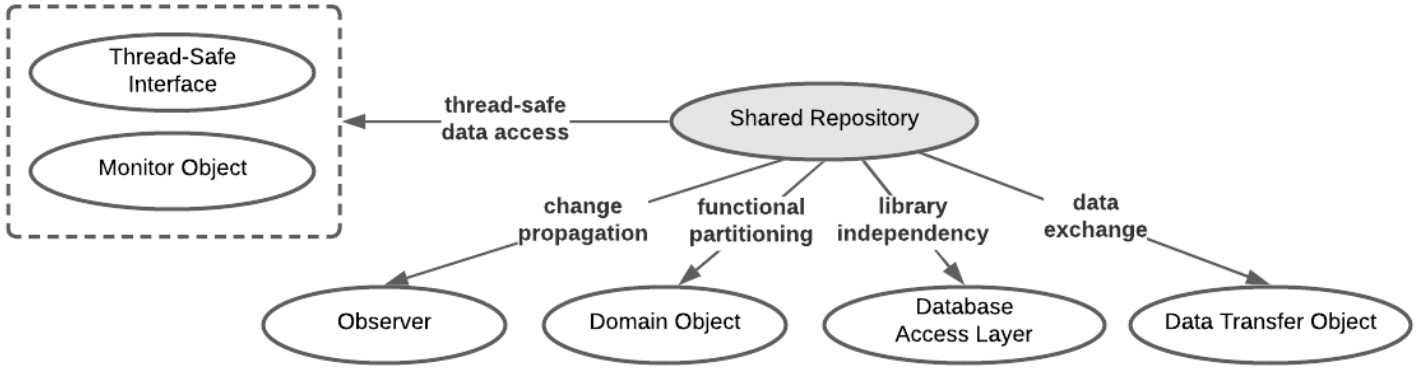
Handlers are usually self-contained and immutable, accepting all necessary data just once via the constructor.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Shared Repository

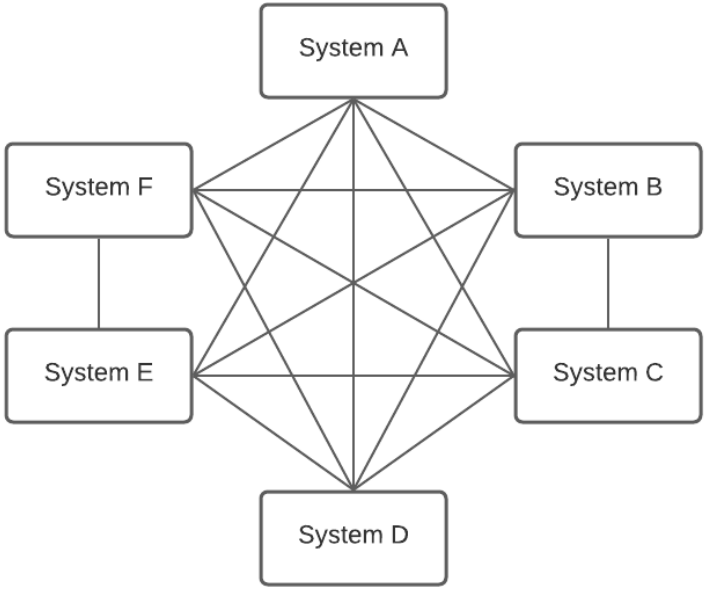
An a**rchitecture pattern language excerpt representing patterns associated with *Shared Repository***.



From Mud to Structure
Shared Repository

Shared Repository is used in the field of inter-system integration. To understand the value of *Shared Repository*, we first look at an alternative approach to interconnecting systems *point-to-point*.

In the point-to-point integration architecture, each system integrates directly with another system to enable their communication and exchange data between the integrated parties.



From Mud to Structure
Shared Repository

In the point-to-point model, when a system A needs to integrate with `n` other systems, system A is responsible for implementing all integration logic related to all `n` systems. Note that integration logic may be bi-directional.

The **integration logic includes e.g., protocol conversions, data transformations, data transfer, which quickly leads to a complex architecture**.

Integrating `n` systems

$$\sum_{k=1}^{n} k = \frac{n\,(n-1)}{2}$$

Integrating `6` systems (`n = 6`)

$$\frac{6\,(6-1)}{2} = 15$$

From Mud to Structure
Shared Repository

In a point-to-point integration architecture, **integrating a new system becomes a problem because the integration logic to the new system must be rewritten** in all the already connected systems.

Therefore, a **point-to-point integration architecture scales very poorly**.

In addition, it **results in very strong coupling between all connected systems**, since each system must bind to the protocol or data transformation specifics of all other systems.

example

From Mud to Structure
Shared Repository

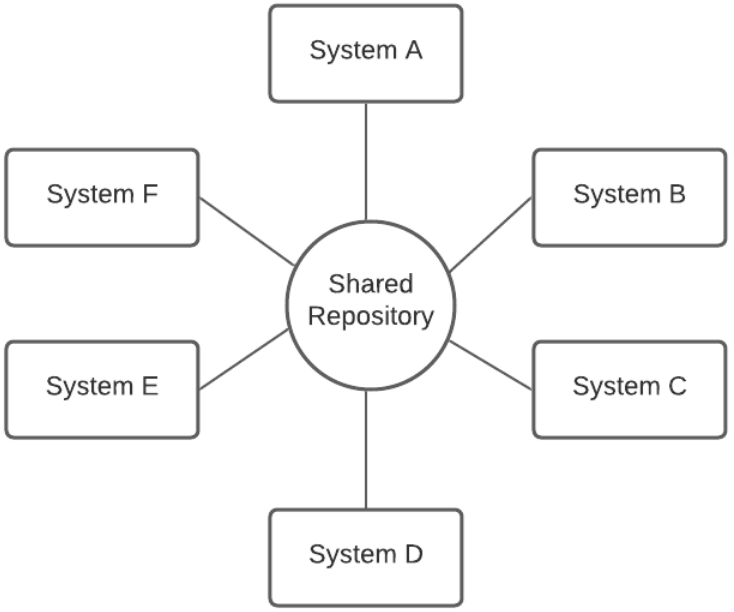
How would you overcome the downsides of many point-2-point connected systems based on a *Shared Repository* – what does such repository need to provide?

example

From Mud to Structure
Shared Repository

The **Shared Repository pattern addresses the shortcomings and drawbacks of the point-to-point model**.

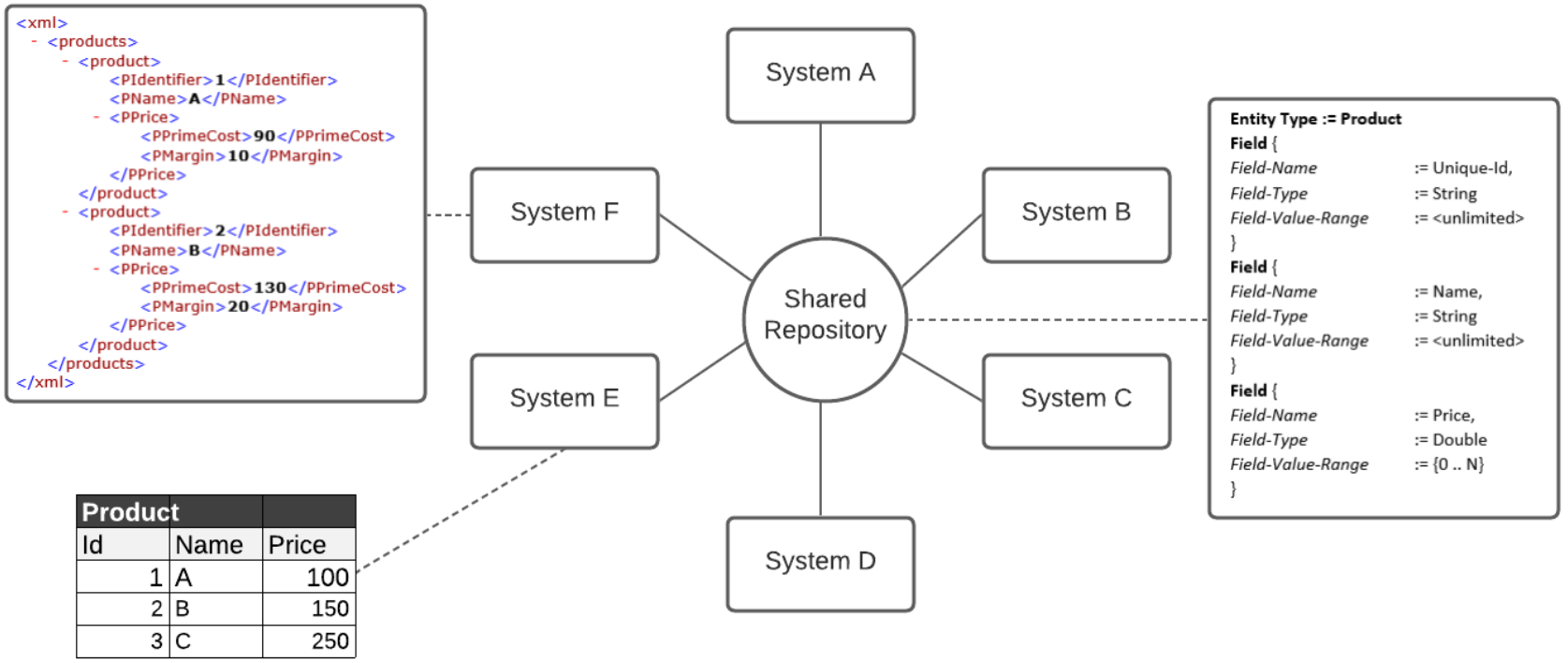
In the *Shared Repository* approach, all systems are **integrated with a centralized *repository* via adapters**. The role of the central broker (i.e., repository) is to either hold a copy of the other systems' data or route traffic to a respectively connected system.



example

From Mud to Structure
Shared Repository

The **repository is thus a central data hub or message mediator and shares data or performs transformations, and routing decisions**.



Product		
Id	Name	Price
1	A	100
2	B	150
3	C	250

From Mud to Structure
Shared Repository

Unlike the point-to-point model, the **Shared Repository pattern scales linearly**.

Repositories typically introduce a **normalized data format as an intermediate format** (i.e., canonical format). To connect a new system to any other system, **only one new connection (to the repository)** needs to be established.

While the *Shared Repository* pattern addresses the problems of the point-to-point integration approach, it **adds few new problems**.

At the **center of these problems are the reliability and performance** of the Repository.

As a centralized system the **Shared Repository becomes a single point of failure, and it must have sufficient performance capacity for the necessary transformations** when there is intensive message exchange between connected systems.

From Mud to Structure
Shared Repository

Several related patterns support, complement or utilize *Shared Repository*:

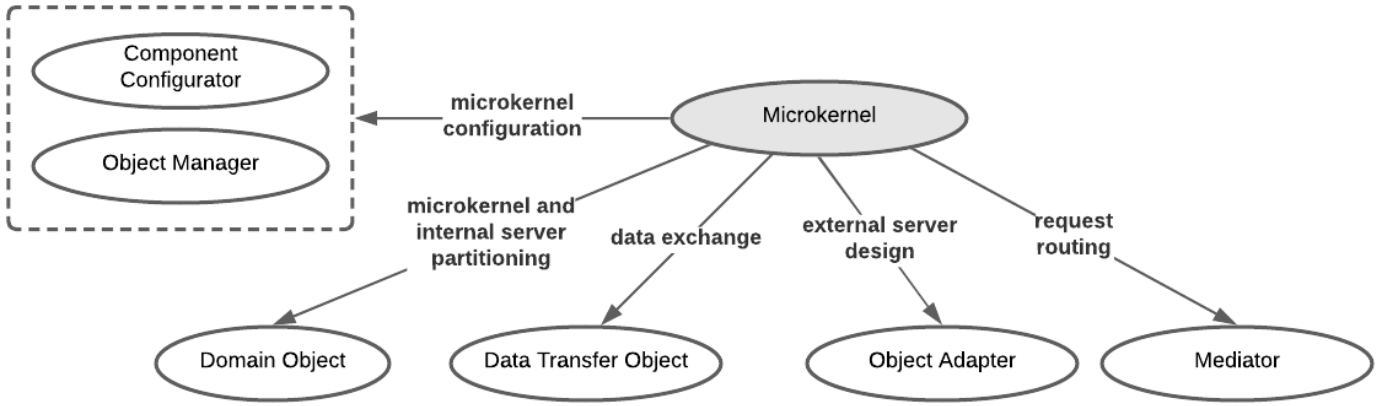
- **Database Access Layer.** While the *Shared Repository* can be as simple as an in-memory data collection, most commonly it is a database external to inter-connecting systems. *Database Access Layer* is used to access respective database systems.
- **Data Transfer Object**. *Data Transfer Objects* help to encapsulate the data passed between the *Shared Repository* and the components of the inter-connected systems.
- **Domain Object**. The data maintained by the *Shared Repository* (i.e., based on a canonical data schema) is often encapsulated inside managed objects (i.e., *Domain Objects*). *Domain Objects* hide the details of concrete data structures and offer meaningful operations for their access and modification. *Domain Objects* allow the connected systems' components to use specific data without becoming dependent on its concrete representation. Domain Objects can also indicate the quality of the represented data via e.g., corresponding quality attributes (e.g., data freshness, reliability, corruptness).
- **Observer**. Many *Shared Repositories* offer a mechanism for notifying application components about data changes within the repository. For example, new data may have been inserted, existing data modified, or data may have been dropped. Components can therefore react immediately to changes to the data in the repository. This change notification mechanism is realized by an *Observer* arrangement: the *Shared Repository* is the subject (aka: Observable) and the connected systems are the *Observers*.

Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Microkernel

An a**rchitecture pattern language excerpt representing patterns associated with *Microkernel.***



From Mud to Structure
Microkernel

Based on a *Microkernel* architecture **you can compose different versions of a system by extending a common but minimal core via a plug-and-play approach**.

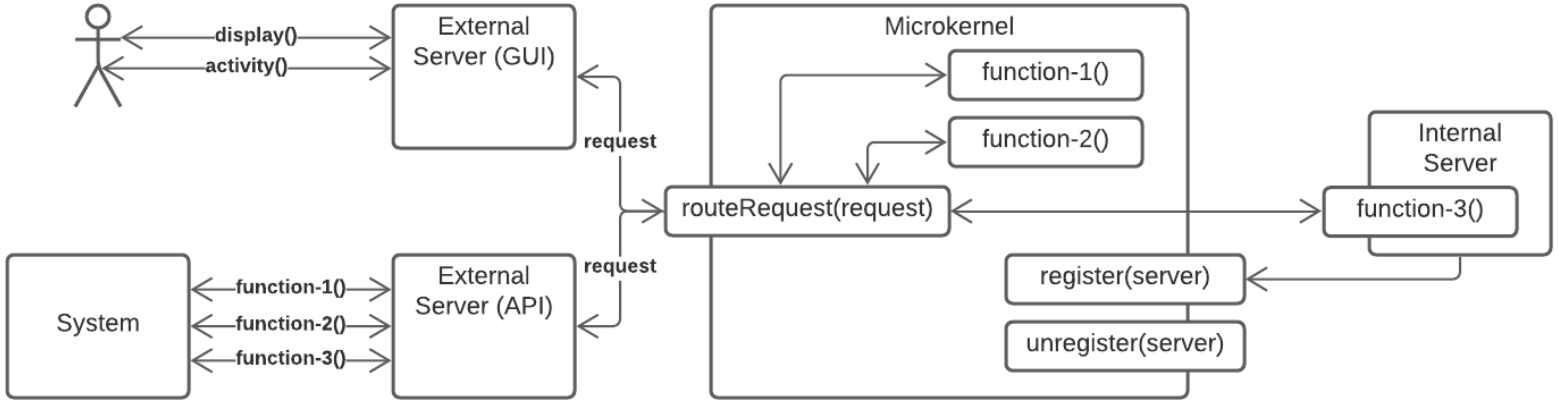
A *Microkernel* **implements the functionality shared by all system versions** and **provides the infrastructure for integrating version-specific functionality, external servers, version-specific user interfaces or APIs**.

You **configure a specific system version by connecting the corresponding internal servers with the microkernel, and providing corresponding external servers to access its functionality**.

Consequently, **all versions of the system share a common functional and infrastructural core, but provide a tailored function set and look-and-feel**.

From Mud to Structure
Microkernel

The **basic design approach of a *Microkernel-based* architecture.**



From Mud to Structure
Microkernel

Clients, whether human or other systems, **access the microkernel's functionality solely via the interfaces or APIs provided by the external servers**.

The **external servers forward all requests they receive to the microkernel**.

If the microkernel implements the requested function itself, it executes the function, otherwise it **routes the request to the corresponding internal server**.

Results are returned accordingly so that the external servers can display or deliver them to the requesting clients.

OS kernels (e.g., Unix kernel) are example of Microkernel architectures.

From Mud to Structure
Microkernel

Several related patterns support, complement or utilize *Microkernel*:

- **Layers.** The internal structure of the *Microkernel* is typically based on Layers. The bottommost layer abstracts from the underlying system platform, thereby supporting *Microkernel* portability. The second layer implements infrastructure functionality, such as resource management, on which the microkernel depends. The layer above hosts the domain functionality that is shared by all system versions. The topmost layer includes the mechanisms for configuring internal servers, as well as for routing requests from external servers to their intended recipient.
- **Domain Object**. Each specific and self-contained function within the microkernel can be realized as a Domain Object, which supports its independent implementation and evolution.
- **Mediator**. The routing functionality of the microkernel is often implemented as a Mediator that receives requests through a uniform interface and dispatches these requests onto corresponding domain functions in the microkernel or the internal servers.
- **Object Adapter**. The design of an external server depends on its complexity and purpose. It can, for example, be a simple Object Adapter that maps the system's published API onto its internal APIs.

example

From Mud to Structure
Microkernel

How does the *Mediator* software design pattern work?

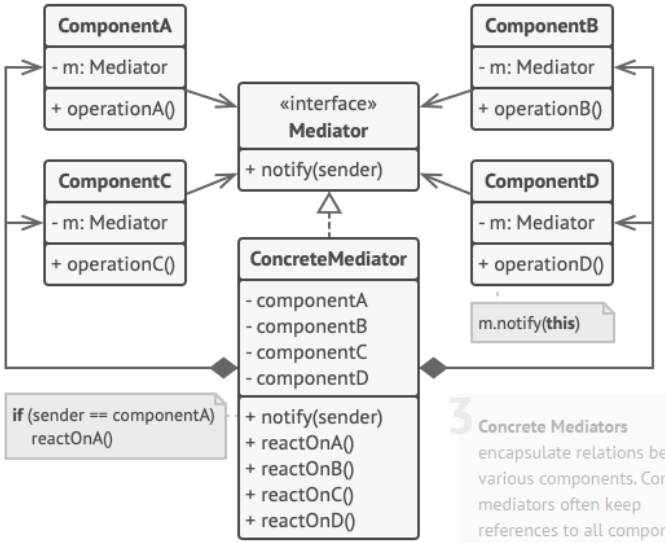
example

From Mud to Structure
Microkernel

Mediator is a behavioural design pattern that lets you reduce chaotic dependencies between objects. The pattern restricts direct communications between the objects and forces them to collaborate only via a mediator object.

1 **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.

2 The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



3 **Concrete Mediators** encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes even manage their lifecycle.

4 Components must not be aware of other components. If something important happens within or to a component, it must only notify the mediator. When the mediator receives the notification, it can easily identify the sender, which might be just enough to decide what component should be triggered in return.

From a component's perspective, it all looks like a total black box. The sender doesn't know who'll end up handling its request, and the receiver doesn't know who sent the request in the first place.

example

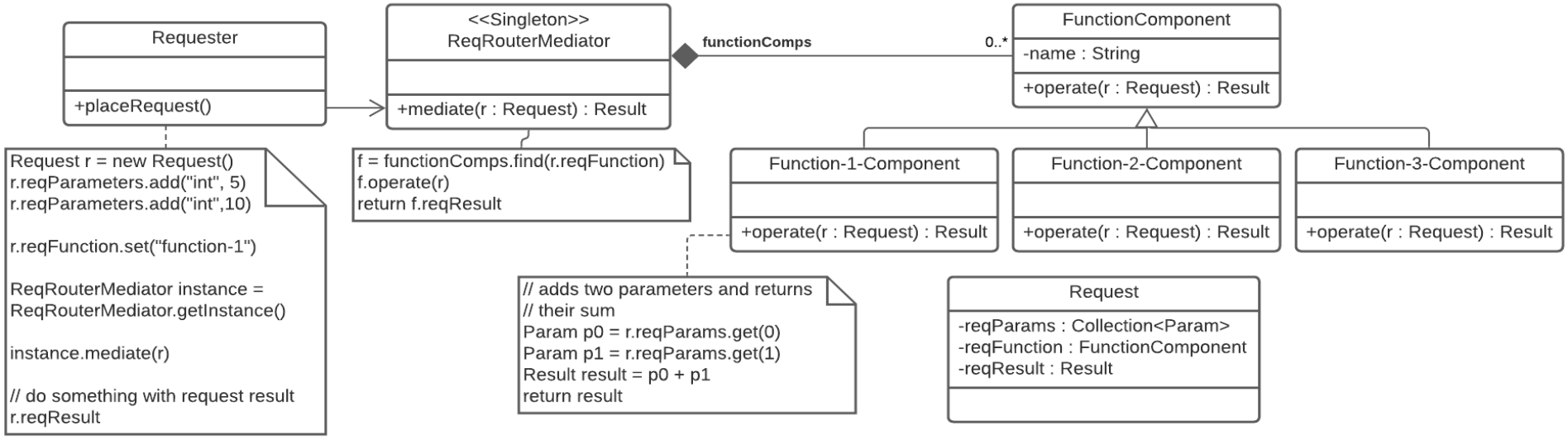
From Mud to Structure
Microkernel

How can we adapt **Mediator** to route requests to the Microkernel's internal and registered servers via a centralized funnel?

example

From Mud to Structure
Microkernel

How can we adapt **Mediator** to route requests to the Microkernel's internal and registered servers via a centralized funnel?

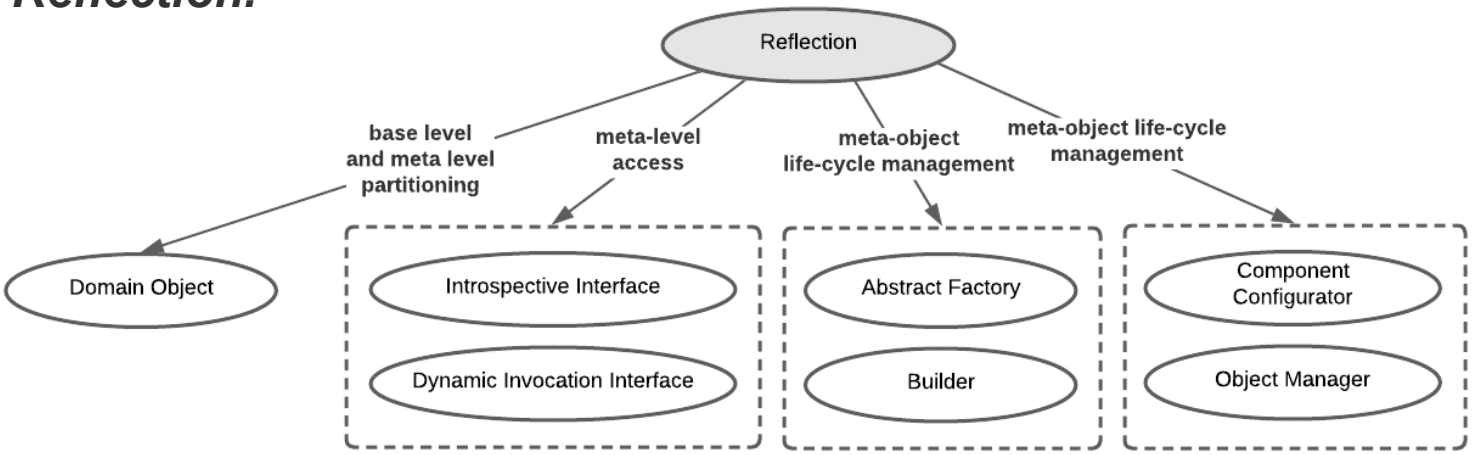


Lecture Agenda

- Pattern
- Architecture Pattern
- From mud to structure
  - Domain Model
  - Domain Object
  - Layers
  - Model-View-Controller
  - Pipes and Filters
  - Shared Repository
  - Microkernel
  - Reflection

From Mud to Structure
Reflection

An a**rchitecture pattern language excerpt representing patterns associated with *Reflection.***



From Mud to Structure
Reflection

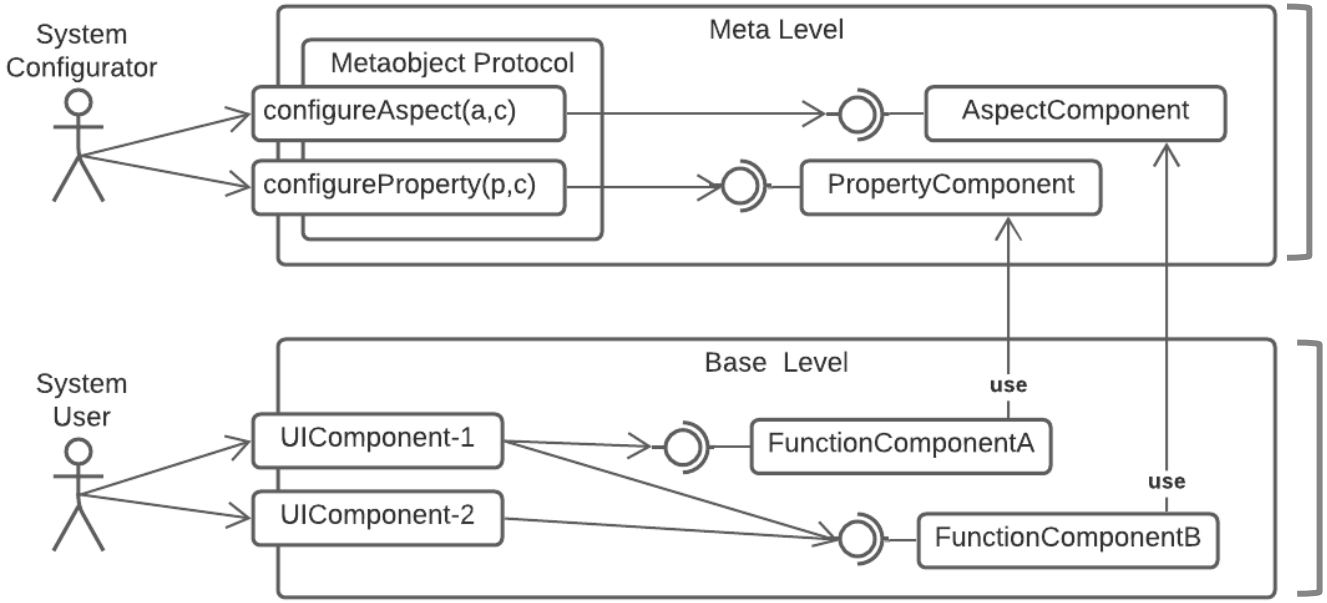
The **Reflection pattern** is used to **objectify information about properties and variant aspects of the application's structure, behaviour, and state into a set of metaobjects**.

Reflection **separates the metaobjects from the core application logic** via a two-layer architecture:

- A meta level contains the metaobjects
- A base level contains the application logic

From Mud to Structure
Reflection

The **basic design approach of a *Reflection-based* architecture.**



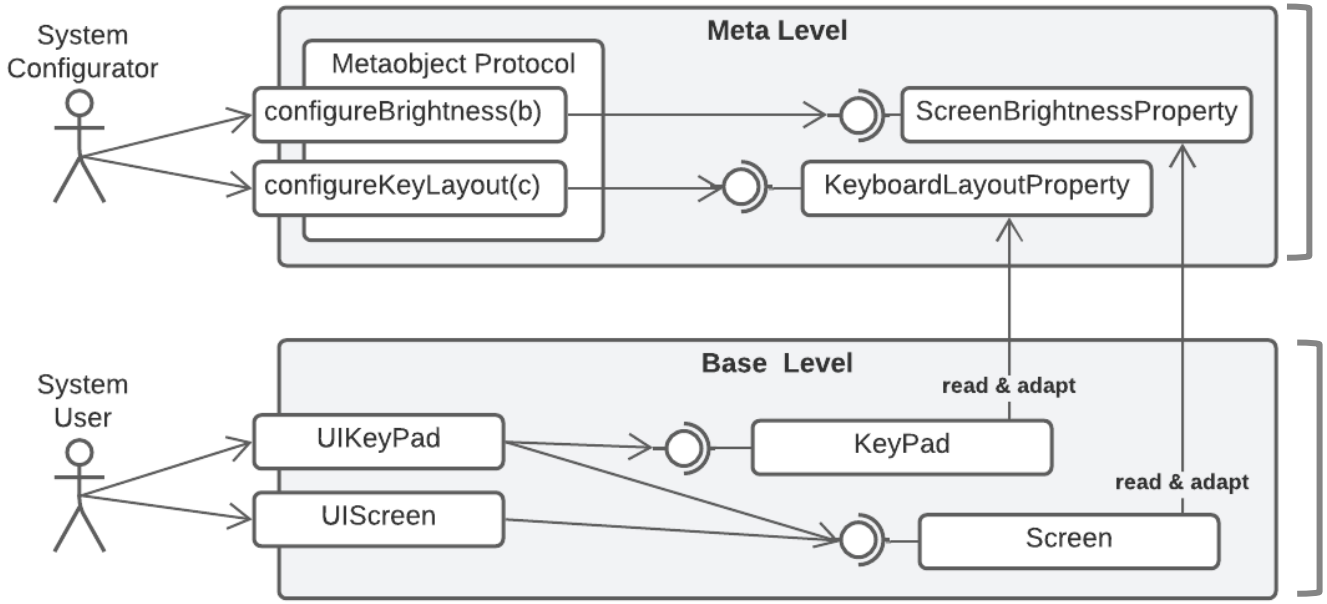
Metaobjects are read and interpreted by Base Level components

Functional components realize application logic by reading and interpreting metaobjects on the Meta Level

example

From Mud to Structure
Reflection

The **basic design approach of a *Reflection-based* architecture.**



Metaobjects are read and interpreted by Base Level components

Functional components realize application logic by reading and interpreting metaobjects on the Meta Level

From Mud to Structure
Reflection

The **Reflection pattern equips the meta level with a metaobject protocol**.

The **metaobject protocol is a specialized interface that system configurators** (i.e., human administrators or even other systems) **can use to dynamically configure and modify all metaobjects**.

The pattern **connects the base level with the meta level such that base-level objects first consult an appropriate metaobject before they execute behaviour** or access state that potentially can vary.

Reflection supports **a high degree of runtime flexibility in a software architecture**. Almost any aspect that can change can be made modifiable.

However, as usual **flexibility comes at its own cost: reflection-based architectures are significantly more complex** to design and maintain.

example

From Mud to Structure
Reflection

What are the steps you suggest to go through to design a system based on the *Reflection* pattern?

example

From Mud to Structure
Reflection

You would follow this procedure to design a *Reflection*-based architecture .

1. Specify a stable design for your application not considering flexibility at all (i.e., stability is the key to flexibility).
2. Encapsulate each self-contained responsibility of the application within a *Domain Object* and make all domain objects the basis of your base level design.
3. Identify all structural and behavioural aspects of the application that can vary.
   a) *Variant behaviour* may include different algorithms for functionality, lifecycle control for domain objects, transaction protocols, IPC mechanisms, and policies for security.
   b) *Variant structure* may include the application's process and thread model, the deployment of domain objects to processes and threads, or even the system's type system.
4. In addition, determine system-wide information, properties, and global state that can influence the behaviour of the application.
5. Realize each variant behavioural and structural aspect and system property as a separate metaobject, and assign all metaobjects to the meta level of the *Reflection* architecture.
6. Open the implementation of each *Domain Object* at the base level such that it consults an appropriate metaobject for each aspect encapsulated in the meta level.
7. To support the creation, configuration, exchange, and disposal of metaobjects at runtime, introduce a metaobject protocol that serves as the sole interface to manage the meta level.

example

From Mud to Structure
Reflection

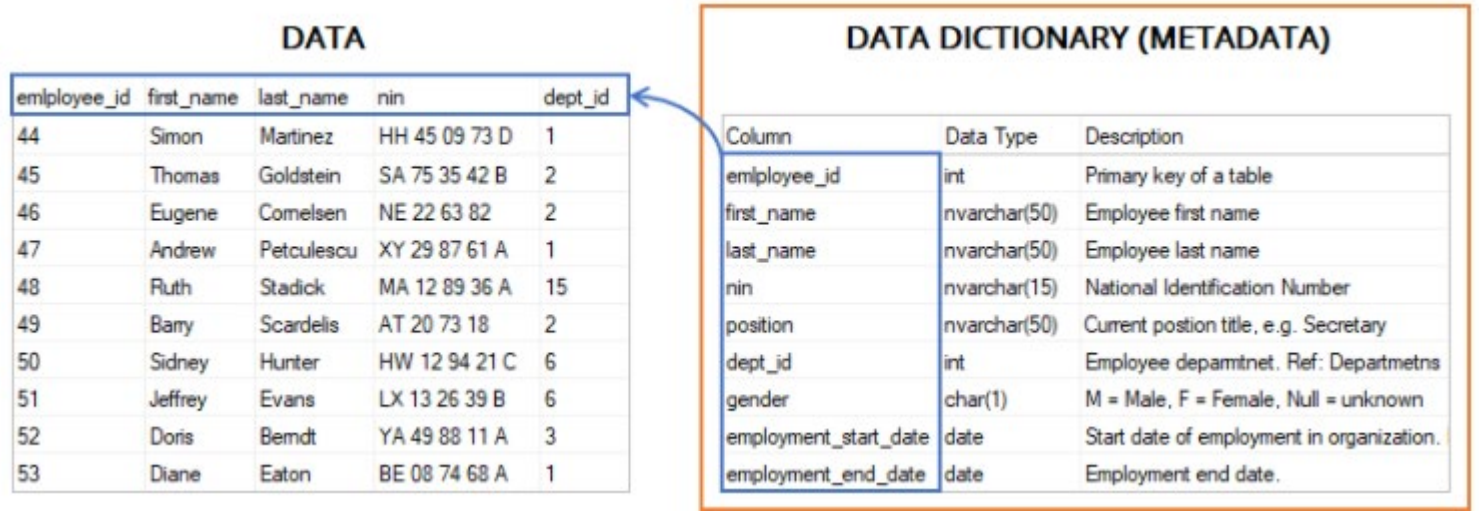
Do you know examples of *Reflection*-based architectures

example

From Mud to Structure
Reflection

Examples of *Reflection*-based architectures

- **Relational database management systems (RDBMSs)** distinguish between user tables system tables. User tables are part of the base level while system tables belong to the meta level (i.e., database dictionary). The metaobject protocol allows you to manipulate the database dictionary (e.g., by creating a new table via SQL's data definition language (DDL) proportions).



DATA

emlployee_id	first_name	last_name	nin	dept_id
44	Simon	Martinez	HH 45 09 73 D	1
45	Thomas	Goldstein	SA 75 35 42 B	2
46	Eugene	Cornelsen	NE 22 63 82	2
47	Andrew	Petculescu	XY 29 87 61 A	1
48	Ruth	Stadick	MA 12 89 36 A	15
49	Barry	Scardelis	AT 20 73 18	2
50	Sidney	Hunter	HW 12 94 21 C	6
51	Jeffrey	Evans	LX 13 26 39 B	6
52	Doris	Berndt	YA 49 88 11 A	3
53	Diane	Eaton	BE 08 74 68 A	1

DATA DICTIONARY (METADATA)

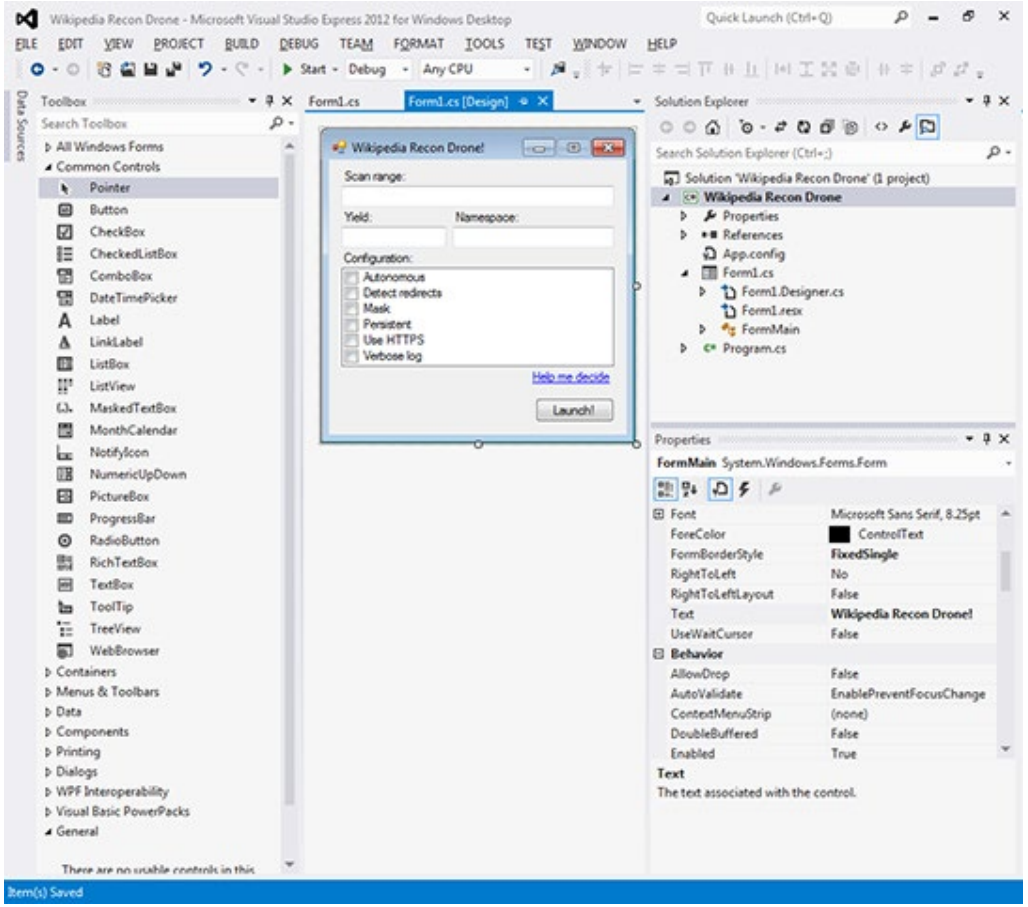
Column	Data Type	Description
emlployee_id	int	Primary key of a table
first_name	nvarchar(50)	Employee first name
last_name	nvarchar(50)	Employee last name
nin	nvarchar(15)	National Identification Number
position	nvarchar(50)	Current postion title, e.g. Secretary
dept_id	int	Employee deparmtnet. Ref: Departmetns
gender	char(1)	M = Male, F = Female, Null = unknown
employment_start_date	date	Start date of employment in organization.
employment_end_date	date	Employment end date.

example

From Mud to Structure
Reflection

Examples of *Reflection*-based architectures

- **Integrated development environments** (IDE) that let you model rich user interface applications allow you to place UI components on forms at design time. The IDEs utilize the component model's metaobject protocol such that they can populate property pages from UI component introspection. A UI component includes both, the base level component logic as well as the meta level aspects and properties to tweak component behaviour.

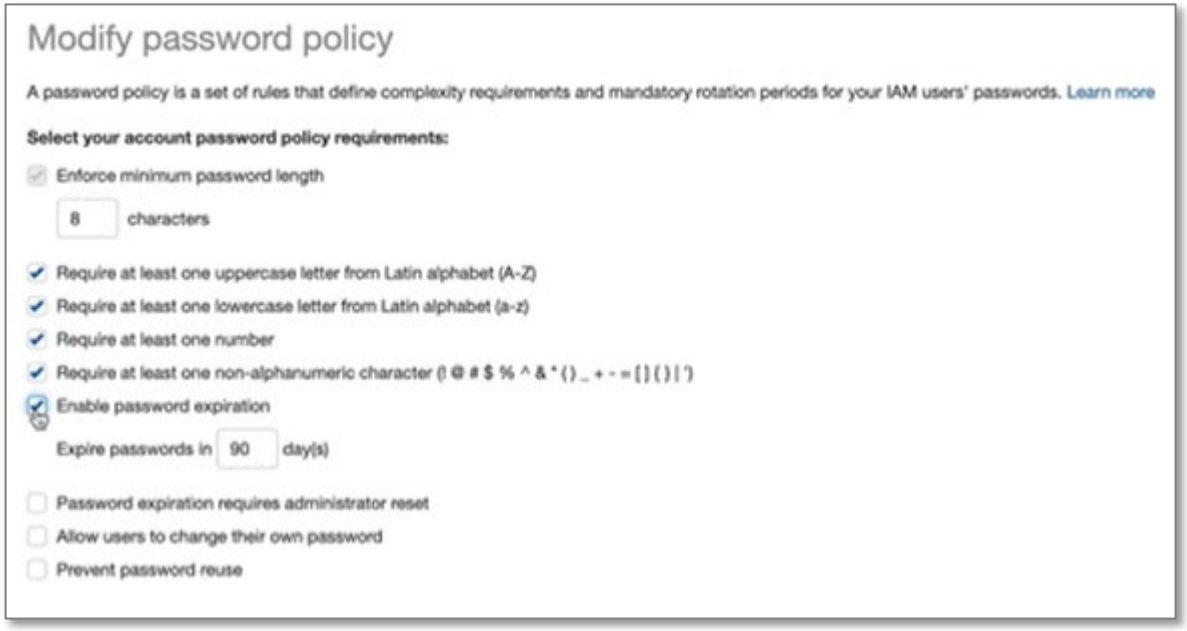


example

From Mud to Structure
Reflection

Examples of *Reflection*-based architectures

- The AWS Password Policy Configurator allows you to define password policy rules that are used by a component responsible for setting and managing passwords to enforce the appropriate policy settings for each user.



From Mud to Structure
Reflection

The metaobject protocol in conjunction with the two-layer structure of a Reflection architecture is a prime example of how the open/close principle can be realized.

The metaobject protocol hides the complexity of software evolution behind a simpler interface, making it easy, uniform, and dynamic (i.e., open) to extend the application, but allows the reflective application to supervise its own evolution so that uncontrolled changes are minimized (i.e., to remain closed).

Several related patterns support, complement or utilize *Reflection*:

- **Abstract Factory, Builder.** To support the creation, configuration, exchange and disposal of metaobjects at runtime, introduce a metaobject protocol that serves as the sole interface to manage the meta level. The metaobject lifecycle infrastructure that is necessary for these activities can be realized with help of Abstract Factory and Builder to create and dispose metaobjects.
- **Component Configurator, Object Manager.** A Component Configurator or Object Manager may help control the execution of specific metaobject lifecycle steps.

Wrap-Up

- **Patterns and Pattern Languages**
  - **Patterns capture proven**, **reusable solutions** to **recurring problems** in a **specific context**.
  - Each **pattern consists** of:
    - **Context** – when the problem occurs.
    - **Problem** – forces and challenges to be resolved.
    - **Solution** – general approach that can be adapted to new contexts.
  - **Purpose**: provide **orientation**, **structure**, and a **common vocabulary** for architects and developers.
  - **Patterns** are **heuristics**, **not recipes** – they guide thinking but do not guarantee success.
  - **Pattern languages connect related patterns** into an **interlinked system** for **solving complex**, **multi-faceted problems**.
    - **Example**: Stairs → Balustrade → Roofing (building design).
    - **In software**: Layers → Domain Object → MVC → Microkernel → Reflection.
  - **Patterns** exist at **different abstraction levels**:
    - **Architecture Patterns** – define global system organization.
    - **Design Patterns** – define component-level structure or collaboration.
    - **Programming Idioms** – define low-level implementation techniques.

Wrap-Up

- **Architecture Patterns – Domain Model**
  - **Captures** the **business logic** and **core responsibilities** of a **system**.
  - **Provides** a **conceptual foundation** for **transforming requirements into architecture**.
  - Often **realized through Domain-Driven Design (DDD)** elements:
    - **Entities** – identity-based domain objects (e.g., Customer).
    - **Value Objects** – immutable objects defined by their properties (e.g., Address).
    - **Aggregates** – consistency boundaries around entities and values.
    - **Services** – stateless operations across multiple domain objects.
    - **Repositories** – access and persistence abstractions.
    - **Factories** – creation of complex objects.
  - **Serves** as a **bridge between business requirements** and **software architecture**.

Wrap-Up

- **Architecture Patterns – Domain Object**
  - **Encapsulates** a **distinct application responsibility** in a **self-contained unit**.
  - **Realization** of the **Domain Model** in **software form** (**similar to Component**).
  - Supports **different granularities**:
    - **Application Service** (e.g., booking, logging)
    - **Functional Component** (e.g., tax calculation)
    - **Domain Entity** (e.g., bank account)
  - Related patterns:
    - **Explicit Interface** – define clear boundaries.
    - **Adapter** / **Bridge** – decouple interface and implementation.
    - **Abstract Factory** / **Builder** – manage creation and lifecycle.

Wrap-Up

- **Architecture Patterns – Layers**
  - The **most common scheme** to **structure software systems**.
  - **Organizes components into layers** of **responsibility** that **depend only** on **lower layers**.
  - **Advantages**
    - **Strict** layering **improves modifiability** and **testability**.
    - **Relaxed** layering **improves performance** and **flexibility**.
  - Typical **criteria** for **partitioning**
    - **Abstraction** (presentation, logic, data)
    - **Rate of change**
    - **Hardware proximity**
  - **Common implementations**: **2-tier**, **3-tier**, **N-tier architectures**.
  - **Related patterns**
    - Domain Object
    - Explicit Interface
    - Observer
    - Command

Wrap-Up

- **Architecture Patterns – Model View Controller (MVC)**
  - **Separates business logic** (**Model**), **user interface** (**View**), and **control flow** (**Controller**).
  - **Enables multiple views** and **reusable models**.
  - **Uses Observer** pattern to **notify views** and **controllers** of **model changes**.
  - **MVC2** (Servlet/JSP) adapts the concept for web applications
    - Servlet = **Controller**
    - JavaBean = **Model**
    - JSP = **View**
  - **Benefits** are **loose coupling**, **easier testing**, and **parallel development**.
  - **Related patterns**
    - Front Controller
    - Page Controller
    - Command
    - Observer
    - Façade

Wrap-Up

- **Architecture Patterns – Pipes and Filters**
  - **Structures data-flow-oriented systems** as **sequences** of **processing steps** (**filters**) **connected by pipes**.
  - Each **filter transforms data incrementally**.
  - **Pipes buffer data between filters**.
  - **Promotes reusability** and **composability** of **processing components**.
  - **Common applications**: **compilers**, **Unix pipelines**, **message brokers**.
  - **Related pattern**
    - Chain of Responsibility.

Wrap-Up

- **Architecture Patterns – Repository**
  - **Centralized data hub** that **connects distributed systems** and **avoids N² point-to-point integrations**.
  - **Uses canonical data formats** and **adapters** for **communication**.
  - **Benefits** are **scalability**, **simplified integration**, **decoupling of systems**.
  - **Drawbacks** are **single point of failure**, **performance bottleneck**.
  - **Related patterns**
    - **Database Access Layer** – structured data access.
    - **Data Transfer Object** – encapsulated data exchange.
    - **Domain Object** – represent canonical data entities.
    - **Observer** – data change notifications to subscribers.

Wrap-Up

- **Architecture Patterns – Microkernel**
  - **Separates** a **minimal**, **stable core** (**microkernel**) **from extensible plug-ins** (**internal/external servers**).
  - **Enables system variation** and **customization via configuration rather than redesign**.
  - **Core handles**
    - Shared functionality
    - Plug-in management
    - Request routing
  - **Examples**: **OS kernels (Unix)**, **IDE platforms**, **business product lines**.
  - **Related patterns**
    - Layers
    - Domain Object
    - Mediator
    - Object Adapter

Wrap-Up

- **Architecture Patterns – Reflection**
  - **Objectifies system structure** and **behavior** in **metaobjects separated** from **base-level logic**.
  - **Supports runtime adaptability** (e.g., policies, algorithms, configurations).
  - **Two-level architecture**
    - **Base level**: executes functional logic.
    - **Meta level**: defines structure and behavior (via metaobject protocol).
  - **Steps** to **design reflective architectures**:
    1. **Identify variant aspects** (e.g., behavior, structure, presentation).
    2. **Represent variants** as metaobjects.
    3. **Base objects consult metaobjects** during **execution**.
    4. **Provide** a **metaobject protocol** (MOP) for **adjusting metaobjects** (even during run-time).
  - **Examples**: **RDBMS data dictionaries**, **IDE property inspectors**, **dynamic policy engines**.
  - **Related patterns**
    - Abstract Factory
    - Builder
    - Component Configurator
    - Object Manager

Questions

Bibliography
Lecture

[Alexander 1977]
Alexander, Christopher; Ishikawa, Sara; Silverstein, Murray; Jacobson, Max; Fiksdahl-King, Ingrid; Angel, Shlomo, A Pattern Language, Oxford University Press, 1977

[Arnold 2021]
Arnold, Ingo, *Enterprise Architecture Function — a pattern language for planning, designing, and executing*, Springer Science and Business Media, Berlin Heidelberg, 2021

[Booch 2009]
Booch, Grady, Object-Oriented Analysis and Design with Applications, Pearson Education, Amsterdam, 2009

[Buschmann et al 2007]
Buschmann, Frank; Henney, Kevlin; Schmidt, Douglas C., *Pattern-Oriented Software Architecture, A Pattern Language for Distributed Computing*, John Wiley & Sons, New York, 2007

Bibliography
Lecture

[Evans 2004]
Evans, Eric; Evans, Eric J., Domain-driven Design - Tackling Complexity in the Heart of Software, Addison-Wesley Professional, Boston, 2004

[Fowler 1997]
Fowler, Martin, Analysis Patterns - Reusable Object Models, Addison-Wesley Professional, Boston, 1997

[Gamma 1994]
Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John, *Design Patterns – Elements of Reusable Object-Oriented Software*, Pearson Education, Amsterdam, 1994

[Matthes et al Pattern Catalogue 2015]
Matthes, Florian, Aleatrati Khosroshahi, Pouya; Hauder, Matheus; Schneider, Alexander, Enterprise Architecture Management Pattern Catalogue, Software Engineering for Business Information Systems (sebis), Technische Universität München, 2015

Bibliography
Lecture

[Vitruvius 2013]
Pollio, Marcus Vitruvius, *De Architectura – Libri Decem*, Elsevier, Amsterdam, 2013